TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN
--------*---------**

BÁO CÁO MÔN HỌC
MÁY HỌC

LAB05. THUẬT TOÁN PHÂN LỚP

Giảng viên hướng dẫn: TS. Đỗ Như Tài

Nhóm sinh viên thực hiện:	**Mã số sinh viên:**
Cao Ngọc Hân	3124411083
Nguyễn An Nhung	3124411203
Trần Đỗ Khánh Linh	3124411151

Thành phố Hồ Chí Minh, tháng 10 năm 2026

LỜI MỞ ĐẦU

Trong thời đại dữ liệu bùng nổ, bài toán phân lớp (classification) xuất hiện ở hầu hết các lĩnh vực: chẩn đoán bệnh trong y tế, dự đoán hành vi khách hàng trong kinh doanh, nhận diện nguy cơ trong nông nghiệp và đời sống. Việc nắm vững các thuật toán phân lớp cơ bản và biết cách áp dụng chúng một cách đúng đắn là nền tảng quan trọng đối với sinh viên ngành Công nghệ Thông tin khi tiếp cận lĩnh vực Học máy (Machine Learning).

Báo cáo "Lab05. Các thuật toán phân lớp" được thực hiện nhằm ôn tập lý thuyết và thực hành ba nhóm giải thuật: Cây quyết định và Rừng cây (Decision Tree & Random Forest), Support Vector Machine (SVM) và Bayes ngây thơ (Naïve Bayes). Với mỗi thuật toán, nhóm em trình bày cơ sở lý thuyết, sau đó áp dụng trên hai bộ dữ liệu thực tế lấy từ Kaggle theo quy trình: khám phá dữ liệu, tiền xử lý, xây dựng mô hình, tối ưu siêu tham số bằng GridSearchCV và đánh giá bằng các chỉ số Accuracy, Precision, Recall, F1-score, ROC AUC. Qua đó, nhóm em phân tích các vấn đề như quá khớp (overfitting), dữ liệu mất cân bằng và đề xuất hướng cải tiến.

Mặc dù đã cố gắng hoàn thành các mục tiêu đề ra, báo cáo chắc chắn không tránh khỏi những hạn chế nhất định. Kính mong nhận được sự đóng góp ý kiến từ Quý Thầy/Cô để bài báo cáo được hoàn thiện hơn.

LỜI CẢM ƠN

Để hoàn thành bài báo cáo thực hành này, nhóm em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy Đỗ Như Tài.

Trong suốt quá trình học tập và thực hiện bài lab, nhờ có những bài giảng tâm huyết, sự hướng dẫn tận tình và những định hướng chuyên môn quý báu của thầy, nhóm em mới có thể tiếp cận các thuật toán phân lớp một cách bài bản, hiểu được bản chất của từng mô hình và hoàn thành báo cáo này.

Do kiến thức chuyên môn và kinh nghiệm thực tiễn của bản thân vẫn còn những giới hạn, bài làm khó tránh khỏi những thiếu sót trong quá trình xử lý dữ liệu, lựa chọn mô hình hay diễn giải kết quả. Kính mong thầy thông cảm và nhận xét, chỉ dẫn để nhóm em rút kinh nghiệm và nâng cao kiến thức trong các bài học tiếp theo.

Em xin kính chúc thầy luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công trong sự nghiệp giáo dục!

Em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIẢNG VIÊN

.....
.....
.....

PHÂN CÔNG

MSSV	Tên	Phân công
3124411083	Cao Ngọc Hân	Nhóm trưởng Mục 2.1 Cây quyết định & Rừng cây (Decision Tree & Random Forest): • Xây dựng & vẽ cây quyết định (DecisionTreeClassifier, graphviz) trên dữ liệu Tín dụng / Titanic. • Tối ưu siêu tham số (GridSearchCV) & Đánh giá mức độ quan trọng đặc trưng (feature_importances_). • Soạn Báo cáo Word (.docx) (Tổng hợp & trả lời lý thuyết : CRISP-DM/Gini/Entropy & Giải thích cây quyết định).
3124411203	Nguyễn An Nhung	Mục 2.2 Support Vector Machine (SVM): • Huấn luyện mô hình SVM & Tìm hàm nhân tối ưu (kernel='linear'/'rbf') trên dữ liệu Iris. • Nhận diện chữ viết tay (MNIST) bằng SVM, vẽ Ma trận nhầm lẫn (Confusion Matrix Heatmap) & classification_report. • Soạn Báo cáo Word (.docx) (Tổng hợp & trả lời lý thuyết : Trích xuất kết quả đánh giá chỉ số Accuracy, Precision, Recall, F1-Score).
3124411151	Trần Đỗ Khánh Linh	Mục 2.3 Bayes ngây thơ (Naïve Bayes) • Tiền xử lý văn bản & Vector hóa dữ liệu (CountVectorizer) cho bài toán Email Spam Detection. • Xây dựng & đánh giá mô hình MultinomialNB trên dữ liệu Spam / Mushroom / Khách hàng. • Thiết kế Slide PowerPoint (.pptx) (Tổng hợp thuật toán, so sánh hiệu năng 3 mô hình & dàn ý thuyết trình).

MỤC LỤC

LỜI MỞ ĐẦU 2
LỜI CẢM ƠN 3
NHẬN XÉT CỦA GIẢNG VIÊN 4
PHÂN CÔNG 5
Chương 1. CÂY QUYẾT ĐỊNH VÀ RỪNG CÂY 7
1.1. Cơ sở lý thuyết 7
1.2. Thư viện và hàm dùng chung 10
1.3. Bài tập thực hành 1: Dữ liệu Titanic 11
1.4. Bài tập thực hành 2: Dữ liệu bệnh tiểu đường 15
1.5. Nhận xét chương 1 19
Chương 2. SUPPORT VECTOR MACHINE (SVM) 20
2.1. Cơ sở lý thuyết 20
2.2. Thư viện và hàm dùng chung 21
2.3. Bài tập thực hành 1: Dữ liệu bệnh tiểu đường 22
2.4. Bài tập thực hành 2: Tình trạng động vật trong rừng 25
2.5. Nhận xét chương 2 27
Chương 3. BAYES NGÂY THƠ (NAÏVE BAYES) 28
3.1. Cơ sở lý thuyết 28
3.2. Thư viện sử dụng 29
3.3. Bài tập thực hành 1: Dự đoán hành vi khách hàng 29
3.4. Bài tập thực hành 2: Phân loại nấm (Mushroom) 32
3.5. Nhận xét chương 3 35
KẾT LUẬN 36
TÀI LIỆU THAM KHẢO 37

Chương 1. CÂY QUYẾT ĐỊNH VÀ RỪNG CÂY

1.1. Cơ sở lý thuyết

1.1.1. Quy trình khai phá dữ liệu CRISP-DM và SEMMA

CRISP-DM (Cross Industry Standard Process for Data Mining) là quy trình chuẩn, không phụ thuộc công cụ, gồm 6 giai đoạn có thể lặp lại: (1) **Business Understanding** – hiểu mục tiêu nghiệp vụ và chuyển thành bài toán khai phá dữ liệu; (2) **Data Understanding** – thu thập, mô tả, khám phá và đánh giá chất lượng dữ liệu; (3) **Data Preparation** – làm sạch, xử lý giá trị thiếu, mã hóa, tạo đặc trưng; (4) **Modeling** – chọn thuật toán và tinh chỉnh tham số; (5) **Evaluation** – đánh giá mô hình theo cả chỉ số kỹ thuật lẫn mục tiêu nghiệp vụ; (6) **Deployment** – triển khai mô hình vào thực tế.

SEMMA (Sample, Explore, Modify, Model, Assess) do SAS Institute đề xuất, tập trung vào phần kỹ thuật: **Sample** – lấy mẫu dữ liệu đại diện; **Explore** – khám phá thống kê và trực quan; **Modify** – biến đổi, chọn và tạo biến; **Model** – xây dựng mô hình; **Assess** – đánh giá độ tin cậy và hiệu quả. Điểm khác chính là SEMMA không có bước hiểu nghiệp vụ và triển khai như CRISP-DM. Trong bài lab này, nhóm em đi theo đúng trình tự: khám phá dữ liệu → tiền xử lý → mô hình hóa → đánh giá.

1.1.2. Cây quyết định hoạt động như thế nào

Cây quyết định (Decision Tree) là mô hình học có giám sát biểu diễn tri thức dưới dạng các luật "nếu – thì" được tổ chức theo cấu trúc cây. Các thành phần chính gồm:

Nút gốc (root node): chứa toàn bộ dữ liệu huấn luyện và là phép kiểm tra đầu tiên trên đặc trưng phân tách tốt nhất.

Nút trong (internal node): mỗi nút là một phép kiểm tra trên một đặc trưng, ví dụ Sex_male <= 0.5.

Nhánh (branch): kết quả của phép kiểm tra (đúng/sai), dẫn tới nút con.

Nút lá (leaf node): không phân tách tiếp, mang nhãn lớp (lớp chiếm đa số) và tỷ lệ các lớp trong nút.

Cây được xây dựng theo kiểu tham lam, từ trên xuống (thuật toán CART trong Scikit-learn): tại mỗi nút, thuật toán thử mọi đặc trưng và mọi ngưỡng, chọn phép tách làm giảm độ "hỗn tạp" nhiều nhất, rồi lặp lại đệ quy cho đến khi gặp điều kiện dừng (đạt max_depth, số mẫu ở nút nhỏ hơn min_samples_split/min_samples_leaf, hoặc nút đã thuần nhất). Khi dự đoán, một mẫu mới đi từ gốc, lần lượt trả lời các câu hỏi ở mỗi nút cho đến khi tới một nút lá, và nhãn của nút lá đó chính là kết quả dự đoán; xác suất dự đoán là tỷ lệ các lớp trong nút lá.

1.1.3. Tiêu chí phân tách: Gini Index, Entropy và Information Gain

Gọi p_i là tỷ lệ mẫu thuộc lớp i trong nút D. Các tiêu chí đo độ hỗn tạp được định nghĩa như sau:

$$Gini(D) = 1 - \Sigma\, p_i^2 \qquad Entropy(D) = -\Sigma\, p_i \cdot log_2(p_i)$$

Information Gain là lượng entropy giảm đi khi tách nút D theo thuộc tính A thành các nút con D_v:

$$IG(D, A) = Entropy(D) - \Sigma\ (|D_v| / |D|) \cdot Entropy(D_v)$$

Cả hai chỉ số đều bằng 0 khi nút thuần nhất và lớn nhất khi các lớp trộn đều (Gini tối đa 0,5 và Entropy tối đa 1 với bài toán 2 lớp). Gini không cần tính logarit nên nhanh hơn và là mặc định của Scikit-learn (criterion='gini'); Entropy/Information Gain (dùng trong ID3, C4.5) nhạy hơn một chút với các lớp hiếm. Trên thực tế, hai tiêu chí thường cho cây gần giống nhau. Information Gain có nhược điểm là thiên vị thuộc tính có nhiều giá trị, vì vậy C4.5 dùng thêm Gain Ratio để chuẩn hóa.

1.1.4. Rừng cây (Random Forest)

Random Forest là phương pháp học kết hợp (ensemble) gồm nhiều cây quyết định. Mỗi cây được huấn luyện trên một mẫu bootstrap (lấy mẫu có hoàn lại từ tập train), và tại mỗi nút chỉ xét một tập con ngẫu nhiên các đặc trưng (max_features, mặc định $\sqrt{p}$ với phân loại). Kết quả cuối cùng là biểu quyết đa số (hoặc trung bình xác suất) của tất cả các cây.

So với một cây đơn lẻ, Random Forest tốt hơn vì một cây sâu có **phương sai cao** – chỉ cần dữ liệu thay đổi nhẹ là cấu trúc cây thay đổi nhiều và dễ quá khớp. Nhờ hai cơ chế ngẫu nhiên (bootstrap và chọn ngẫu nhiên đặc trưng), các cây trong rừng ít tương quan với nhau, nên khi lấy trung bình thì sai số do phương sai giảm mạnh trong khi độ chệch hầu như không tăng. Ngoài ra rừng cây còn cho ước lượng lỗi out-of-bag (OOB) mà không cần tập validation riêng.

1.1.5. Ưu điểm và hạn chế

Cây quyết định – ưu điểm: dễ hiểu, dễ trực quan hóa, giải thích được bằng luật; xử lý được cả biến số và biến phân loại; không cần chuẩn hóa dữ liệu; huấn luyện nhanh. Hạn chế: rất dễ quá khớp nếu để cây phát triển sâu; không ổn định (phương sai cao); ranh giới quyết định chỉ gồm các đoạn song song trục nên khó xấp xỉ quan hệ chéo/tuyến tính trơn; thiên lệch về lớp đa số khi dữ liệu mất cân bằng.

Random Forest – ưu điểm: độ chính xác cao và ổn định, ít quá khớp hơn, cho biết mức độ quan trọng của đặc trưng, ít phải tinh chỉnh. Hạn chế: khó diễn giải hơn một cây, tốn bộ nhớ và thời gian khi số cây lớn, dự đoán chậm hơn.

Cây quyết định thường hoạt động kém khi: dữ liệu ít mà số đặc trưng nhiều; quan hệ giữa đặc trưng và nhãn là tuyến tính/trơn; dữ liệu nhiễu nhiều; các lớp mất cân bằng nặng; hoặc khi cần ngoại suy ra ngoài miền giá trị đã thấy.

1.1.6. Các bước xây dựng cây quyết định và Random Forest bằng Scikit-learn

Quy trình chung nhóm em áp dụng gồm các bước:

Bước 1: Nạp dữ liệu bằng pandas, khám phá phân bố và giá trị thiếu.

Bước 2: Tiền xử lý – điền giá trị thiếu, mã hóa biến phân loại (One-Hot bằng pd.get_dummies), bỏ các cột định danh.

Bước 3: Chia dữ liệu train/test bằng train_test_split(test_size=0.2, stratify=y).

Bước 4: Khởi tạo DecisionTreeClassifier(max_depth=…) hoặc RandomForestClassifier(n_estimators=…, max_depth=…, max_features='sqrt') rồi gọi .fit(X_train, y_train).

Bước 5: Dự đoán bằng .predict() / .predict_proba() và đánh giá bằng accuracy_score, roc_auc_score, classification_report, confusion_matrix.

Các tham số thường dùng của Random Forest: **n_estimators** (số cây), **max_depth** (độ sâu tối đa mỗi cây), **min_samples_split**, **min_samples_leaf** (số mẫu tối thiểu để tách/ở lá), **max_features** (số đặc trưng xét ở mỗi nút), **bootstrap**, **class_weight** (xử lý mất cân bằng) và **random_state**.

1.1.7. Đánh giá tầm quan trọng của đặc trưng

Sau khi huấn luyện, thuộc tính **feature_importances_** của Random Forest cho biết mức giảm độ hỗn tạp (Mean Decrease in Impurity) trung bình mà mỗi đặc trưng đóng góp trên tất cả các cây, đã chuẩn hóa để tổng bằng 1. Ta đưa vào pd.Series với chỉ số là tên cột rồi vẽ biểu đồ cột ngang. Cần lưu ý cách tính này thiên vị các biến liên tục/nhiều giá trị; khi cần kết quả khách quan hơn có thể dùng permutation_importance của sklearn.inspection.

1.1.8. Điều chỉnh siêu tham số với GridSearchCV và RandomizedSearchCV

GridSearchCV thử toàn bộ tổ hợp trong lưới tham số (param_grid), mỗi tổ hợp được đánh giá bằng kiểm định chéo k-fold (cv=5): tập train chia thành k phần, lần lượt lấy 1 phần làm validation và k−1 phần còn lại để huấn luyện, điểm cuối cùng là trung bình k lần. Tham số scoring chọn chỉ số tối ưu (ví dụ 'roc_auc', 'accuracy'); return_train_score=True giúp so sánh điểm train và validation để phát hiện quá khớp; refit=True huấn luyện lại mô hình tốt nhất trên toàn bộ tập train (best_estimator_). **RandomizedSearchCV** chỉ lấy ngẫu nhiên n_iter tổ hợp từ các phân phối tham số, phù hợp khi không gian tìm kiếm lớn vì tiết kiệm thời gian mà vẫn cho kết quả gần tối ưu.

1.1.9. Các chỉ số đánh giá mô hình phân lớp

Từ ma trận nhầm lẫn (TP, TN, FP, FN), các chỉ số được dùng xuyên suốt báo cáo gồm:

Accuracy = (TP + TN) / tổng số mẫu – tỷ lệ dự đoán đúng; dễ gây hiểu nhầm khi dữ liệu mất cân bằng.

Precision = TP / (TP + FP) – trong các mẫu dự đoán là dương, bao nhiêu mẫu đúng.

Recall = TP / (TP + FN) – trong các mẫu dương thật, mô hình bắt được bao nhiêu.

F1-score = 2 · Precision · Recall / (Precision + Recall) – trung bình điều hòa của hai chỉ số trên.

ROC AUC – diện tích dưới đường cong ROC (TPR theo FPR khi thay đổi ngưỡng), đo khả năng xếp hạng mẫu dương cao hơn mẫu âm, không phụ thuộc ngưỡng 0,5.

1.2. Thư viện và hàm dùng chung

Nhóm em sử dụng numpy, pandas để xử lý dữ liệu; matplotlib, seaborn để trực quan hóa; các lớp DecisionTreeClassifier, plot_tree, RandomForestClassifier, GridSearchCV và các hàm đánh giá từ Scikit-learn. Hàm find_file giúp tự tìm file CSV ở thư mục hiện tại hoặc trong /kaggle/input khi chạy trên Kaggle.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
import warnings; warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, roc_auc_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

import glob, os
def find_file(patterns):
    """Tìm file CSV ở thư mục hiện tại hoặc trong /kaggle/input (khi chạy trên Kaggle)."""
    for p in patterns:
        hits = glob.glob(p) + glob.glob('/kaggle/input/**/' + p, recursive=True)
        if hits: return hits[0]
    raise FileNotFoundError('Không thấy file ' + str(patterns) + ' - hãy tải từ Kaggle và đặt cạnh notebook.')
```

Để hai bài tập dùng chung một quy trình, nhóm em viết hàm **run_tree_forest(X, y, name)** thực hiện 5 bước: chia train/test 80:20 có stratify; vẽ cây nông max_depth=3 để đọc luật; GridSearchCV cho cây theo max_depth và min_samples_leaf (scoring ROC AUC, cv=5) kèm đồ thị train/validation theo độ sâu; GridSearchCV cho Random Forest theo n_estimators và max_depth; cuối cùng đánh giá trên tập test và vẽ feature importance.

```
def run_tree_forest(X, y, name):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
    print(f'[{name}] train={X_train.shape}, test={X_test.shape}, tỷ lệ lớp 1 = {y.mean():.3f}')

    # 1) Cây nông để trực quan hóa
    dt_small = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_train, y_train)
    plt.figure(figsize=(16, 7))
    plot_tree(dt_small, feature_names=list(X.columns), class_names=['0', '1'],
              filled=True, rounded=True, proportion=True, fontsize=8)
    plt.title(f'{name} - cây quyết định max_depth=3'); plt.show()

    # 2) GridSearchCV cho cây quyết định
    dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                           {'max_depth': [1, 2, 3, 4, 5, 6, 8, 10, 12], 'min_samples_leaf': [1, 5, 10, 20]},
                           scoring='roc_auc', cv=5, return_train_score=True)
    dt_grid.fit(X_train, y_train)
    print('Cây - tham số tốt nhất:', dt_grid.best_params_, '| ROC AUC CV = %.4f' % dt_grid.best_score_)
    res = pd.DataFrame(dt_grid.cv_results_)
    res = res[res['param_min_samples_leaf'] == dt_grid.best_params_['min_samples_leaf']]
    plt.figure(figsize=(6, 4))
    plt.plot(res['param_max_depth'].astype(int), res['mean_train_score'], '-o', label='train')
    plt.plot(res['param_max_depth'].astype(int), res['mean_test_score'], '-o', label='validation')
    plt.xlabel('max_depth'); plt.ylabel('ROC AUC'); plt.legend(); plt.title(f'{name} - quá khớp theo độ sâu'); plt.show()

    # 3) GridSearchCV cho Random Forest
    rf_grid = GridSearchCV(RandomForestClassifier(random_state=42, n_jobs=-1),
                           {'n_estimators': [10, 50, 100, 200], 'max_depth': [3, 5, 8, 12]},
                           scoring='roc_auc', cv=5)
    rf_grid.fit(X_train, y_train)
    print('RF  - tham số tốt nhất:', rf_grid.best_params_, '| ROC AUC CV = %.4f' % rf_grid.best_score_)

    # 4) Đánh giá trên tập test
    rows = []
    fig, axs = plt.subplots(1, 2, figsize=(9, 3.5))
    for ax, (label, m) in zip(axs, [('Decision Tree', dt_grid.best_estimator_), ('Random Forest', rf_grid.best_estimator_)]):
        pred = m.predict(X_test)
        rows.append({'Model': label, 'Accuracy test': accuracy_score(y_test, pred),
                     'ROC AUC train': roc_auc_score(y_train, m.predict_proba(X_train)[:, 1]),
                     'ROC AUC test': roc_auc_score(y_test, m.predict_proba(X_test)[:, 1])})
        ConfusionMatrixDisplay(confusion_matrix(y_test, pred)).plot(ax=ax, colorbar=False, cmap='Blues')
        ax.set_title(label)
        print('\n=== ' + label + ' ===\n' + classification_report(y_test, pred))
    plt.tight_layout(); plt.show()

    # 5) Độ quan trọng của đặc trưng (Random Forest)
    imp = pd.Series(rf_grid.best_estimator_.feature_importances_, index=X.columns).sort_values()
    imp.plot.barh(figsize=(6, max(3, 0.3 * len(imp)))); plt.title(f'{name} - feature importance (RF)'); plt.show()
    return pd.DataFrame(rows).round(4), imp.sort_values(ascending=False)
```

1.3. Bài tập thực hành 1: Dữ liệu Titanic

Mục tiêu: dự đoán hành khách có sống sót (Survived = 1) dựa trên hạng vé, giới tính, tuổi, số người thân đi cùng, giá vé và cảng lên tàu.

1.3.1. Nạp dữ liệu và khám phá

```
titanic = pd.read_csv(find_file(['train.csv', 'test.csv']))
print(titanic.shape)
titanic.head()
```

```
(891, 12)
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

Hình 1.1. Kích thước và 5 dòng đầu của dữ liệu Titanic

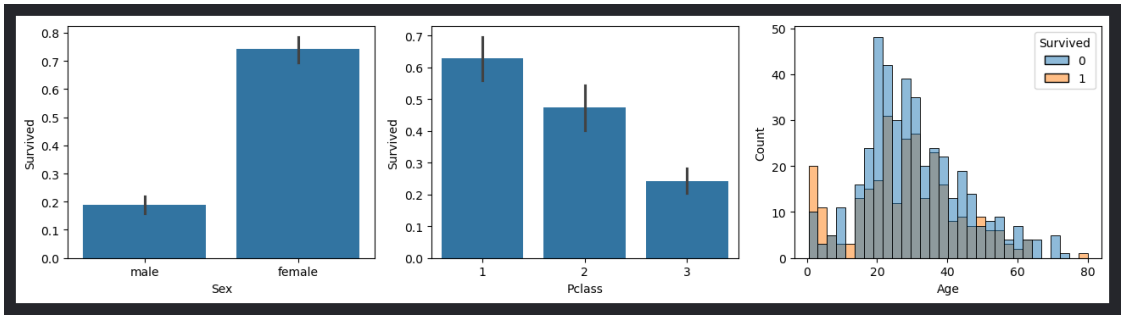
Tập dữ liệu có 891 hành khách và 12 cột, trong đó PassengerId, Name, Ticket là cột định danh không có ý nghĩa dự đoán trực tiếp.

```
print(titanic.isna().sum()[lambda s: s > 0])        # cột bị thiếu
print('\nTỷ lệ sống sót:', titanic['Survived'].mean().round(3))
fig, axs = plt.subplots(1, 3, figsize=(13, 3.5))
sns.barplot(data=titanic, x='Sex', y='Survived', ax=axs[0])
sns.barplot(data=titanic, x='Pclass', y='Survived', ax=axs[1])
sns.histplot(data=titanic, x='Age', hue='Survived', bins=30, ax=axs[2])
plt.tight_layout(); plt.show()
```

```
Age         177
Cabin       687
Embarked      2
dtype: int64

Tỷ lệ sống sót: 0.384
```

Hình 1.2. Số giá trị thiếu và tỷ lệ sống sót



Hình 1.3. Tỷ lệ sống sót theo giới tính, hạng vé và phân bố tuổi

Nhận xét: Age thiếu 177 giá trị (~20%), Cabin thiếu 687 (~77%), Embarked thiếu 2. Tỷ lệ sống sót chung là 38,4%, tức dữ liệu hơi mất cân bằng. Nữ giới sống sót khoảng 74% trong khi nam chỉ khoảng 19%; hạng 1 sống sót ~63%, hạng 3 chỉ ~24%; trẻ nhỏ dưới 5 tuổi có tỷ lệ sống sót cao. Đây là dấu hiệu cho thấy Sex, Pclass và Age sẽ là các đặc trưng quan trọng.

1.3.2. Tiền xử lý dữ liệu

Age được điền bằng trung vị theo nhóm (Sex, Pclass) vì tuổi khác nhau rõ giữa các nhóm; Embarked điền bằng mode. Cabin thiếu quá nhiều nên được chuyển thành biến nhị phân HasCabin. Nhóm em tạo thêm FamilySize = SibSp + Parch + 1 và IsAlone, bỏ PassengerId, Name, Ticket, Cabin rồi mã hóa One-Hot cho Sex, Embarked. Mô hình cây không cần chuẩn hóa dữ liệu số vì phép tách chỉ dựa trên ngưỡng.

```
df = titanic.copy()
df['Age'] = df['Age'].fillna(df.groupby(['Sex', 'Pclass'])['Age'].transform('median')).fillna(df['Age'].median())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
df['HasCabin'] = df['Cabin'].notna().astype(int)
df['FamilySize'] = df['SibSp'] + df['Parch'] + 1
df['IsAlone'] = (df['FamilySize'] == 1).astype(int)
df = df.drop(columns=[c for c in ['PassengerId', 'Name', 'Ticket', 'Cabin'] if c in df.columns])
df = pd.get_dummies(df, columns=['Sex', 'Embarked'], drop_first=True, dtype=int)
X1, y1 = df.drop(columns='Survived'), df['Survived']
print(X1.shape, '| còn thiếu:', int(X1.isna().sum().sum()))
X1.head()
```

```
(891, 11) | còn thiếu: 0
   Pclass   Age  SibSp  Parch     Fare  HasCabin  FamilySize  IsAlone  \
0       3  22.0      1      0   7.2500         0           2        0
1       1  38.0      1      0  71.2833         1           2        0
2       3  26.0      0      0   7.9250         0           1        1
3       1  35.0      1      0  53.1000         1           2        0
4       3  35.0      0      0   8.0500         0           1        1

   Sex_male  Embarked_Q  Embarked_S
0         1           0           1
1         0           0           0
2         0           0           1
3         0           0           1
4         1           0           1
```

Hình 1.4. Dữ liệu Titanic sau tiền xử lý

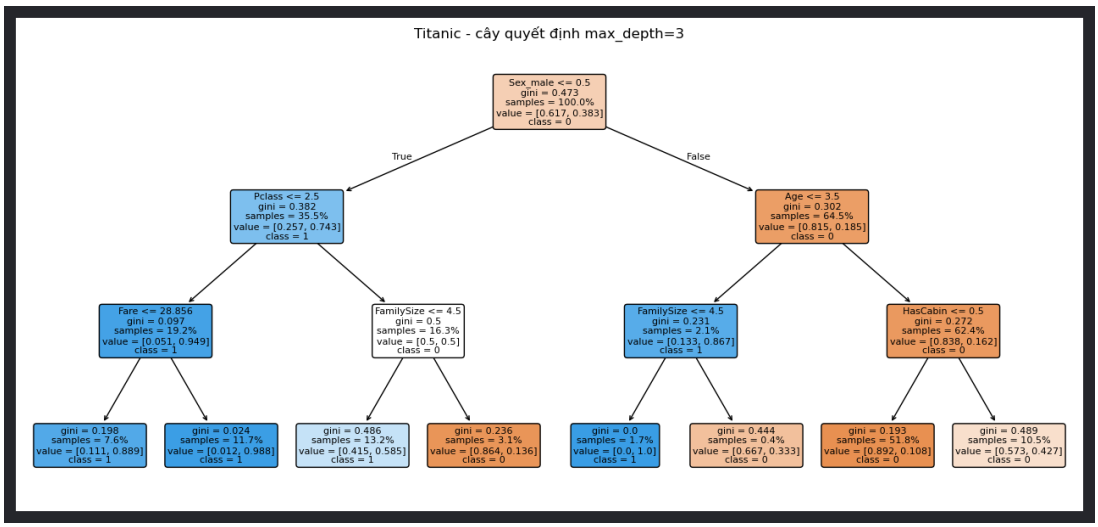
Sau xử lý, X1 có 891 dòng, 11 đặc trưng và không còn giá trị thiếu.

1.3.3. Xây dựng, tối ưu và đánh giá mô hình

```
res1, imp1 = run_tree_forest(X1, y1, 'Titanic')
res1
```

```
[Titanic] train=(712, 11), test=(179, 11), tỷ lệ lớp 1 = 0.384
```

Hình 1.5. Kích thước tập train/test của Titanic

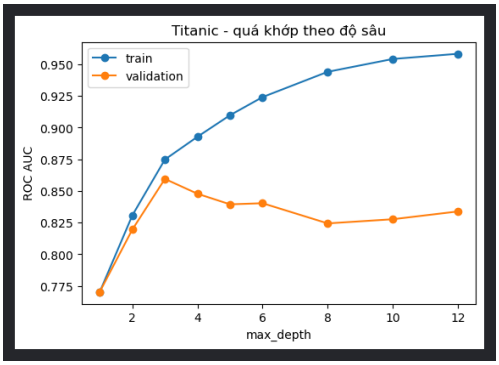


Hình 1.6. Cây quyết định max_depth=3 trên dữ liệu Titanic

Đọc cây: nút gốc tách theo Sex_male, đúng với nhận định khi khám phá dữ liệu. Nhánh nữ (Sex_male <= 0.5) có 74,3% sống sót; trong đó nữ hạng 1–2 có giá vé trên 28,86 sống sót tới 98,8%. Nhánh nam có tỷ lệ sống sót 18,5%, ngoại trừ bé trai dưới 3,5 tuổi trong gia đình nhỏ (FamilySize <= 4.5) sống sót 100%. Luật này phản ánh nguyên tắc "phụ nữ và trẻ em trước" trong thực tế.

```
Cây - tham số tốt nhất: {'max_depth': 3, 'min_samples_leaf': 5} | ROC AUC CV = 0.8594
```

Hình 1.7. Tham số tốt nhất của cây quyết định



Hình 1.8. ROC AUC train/validation theo độ sâu của cây (Titanic)

Phân tích quá khớp: khi max_depth tăng, ROC AUC trên train tăng liên tục tới ~0,96 còn validation đạt đỉnh 0,859 tại max_depth = 3 rồi giảm xuống ~0,82–0,83. Khoảng cách train – validation ngày càng lớn chứng tỏ cây sâu đang "học thuộc" dữ liệu. GridSearchCV chọn max_depth = 3, min_samples_leaf = 5.

```
RF  - tham số tốt nhất: {'max_depth': 5, 'n_estimators': 50} | ROC AUC CV = 0.8744

=== Decision Tree ===
              precision    recall  f1-score   support

           0       0.80      0.87      0.83       110
           1       0.76      0.65      0.70        69

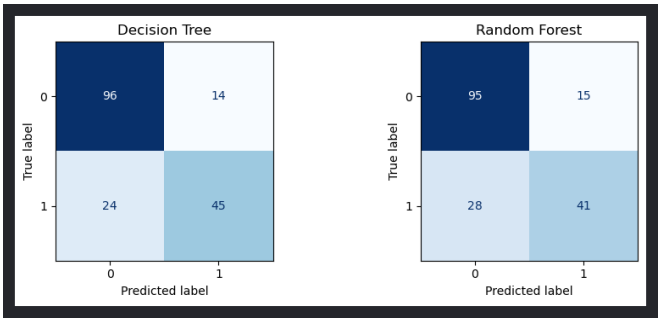
    accuracy                           0.79       179
   macro avg       0.78      0.76      0.77       179
weighted avg       0.79      0.79      0.78       179


=== Random Forest ===
              precision    recall  f1-score   support

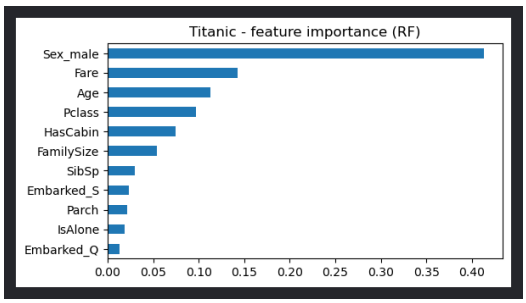
           0       0.77      0.86      0.82       110
           1       0.73      0.59      0.66        69

    accuracy                           0.76       179
   macro avg       0.75      0.73      0.74       179
weighted avg       0.76      0.76      0.75       179
```

Hình 1.9. Tham số tốt nhất của Random Forest và báo cáo phân loại trên tập test



Hình 1.10. Ma trận nhầm lẫn của Decision Tree và Random Forest (Titanic)



Hình 1.11. Mức độ quan trọng của đặc trưng theo Random Forest (Titanic)

	Model	Accuracy test	ROC AUC train	ROC AUC test
0	Decision Tree	0.7877	0.872	0.8043
1	Random Forest	0.7598	0.920	0.8373

Hình 1.12. Bảng tổng hợp kết quả trên tập test (Titanic)

Nhận xét: Random Forest (max_depth = 5, n_estimators = 50) có ROC AUC CV 0,874 và ROC AUC test 0,837, cao hơn cây quyết định (0,804). Ở ngưỡng mặc định 0,5, cây quyết định đạt accuracy 0,788 và rừng cây đạt 0,760; hai mô hình có ưu thế khác nhau: rừng cây phân biệt xác suất giữa hai lớp tốt hơn (ROC AUC cao hơn), còn cây quyết định có recall lớp sống sót nhỉnh hơn (0,65 so với 0,59). Đây là kết quả tốt đối với bộ dữ liệu Titanic chỉ gồm 891 hành khách. Feature importance xác nhận Sex_male là đặc trưng quan trọng nhất (~0,41), tiếp theo là Fare, Age, Pclass, HasCabin.

1.4. Bài tập thực hành 2: Dữ liệu bệnh tiểu đường

Mục tiêu: dự đoán nhãn diabetes (1 = mắc tiểu đường) từ các thuộc tính nhân khẩu học, tiền sử hút thuốc và chỉ số sức khỏe trong diabetes_prediction_dataset.csv.

1.4.1. Nạp dữ liệu và khám phá

```
diab = pd.read_csv(find_file(['diabetes_prediction_dataset.csv']))
print(diab.shape)
display_cols = diab.describe(include='all').T
print(display_cols)
print('\nTỷ lệ diabetes=1:', diab['diabetes'].mean().round(3))
print('\nPhân bố nhãn:')
print(diab['diabetes'].value_counts())
```

```
(100000, 9)
                     count unique      top   freq       mean        std  \
gender              100000      3   Female  58552        NaN        NaN
age                100000.0    NaN      NaN    NaN  41.885856   22.51684
hypertension       100000.0    NaN      NaN    NaN    0.07485    0.26315
heart_disease      100000.0    NaN      NaN    NaN    0.03942   0.194593
smoking_history       100000      6  No Info  35816        NaN        NaN
bmi                100000.0    NaN      NaN    NaN  27.320767   6.636783
HbA1c_level        100000.0    NaN      NaN    NaN   5.527507   1.070672
blood_glucose_level  100000.0    NaN      NaN    NaN  138.05806  40.708136
diabetes           100000.0    NaN      NaN    NaN      0.085   0.278883

                      min    25%    50%    75%    max
gender                NaN    NaN    NaN    NaN    NaN
age                  0.08   24.0   43.0   60.0   80.0
hypertension          0.0    0.0    0.0    0.0    1.0
heart_disease         0.0    0.0    0.0    0.0    1.0
smoking_history       NaN    NaN    NaN    NaN    NaN
bmi                 10.01  23.63  27.32  29.58  95.69
HbA1c_level           3.5    4.8    5.8    6.2    9.0
blood_glucose_level  80.0  100.0  140.0  159.0  300.0
diabetes              0.0    0.0    0.0    0.0    1.0

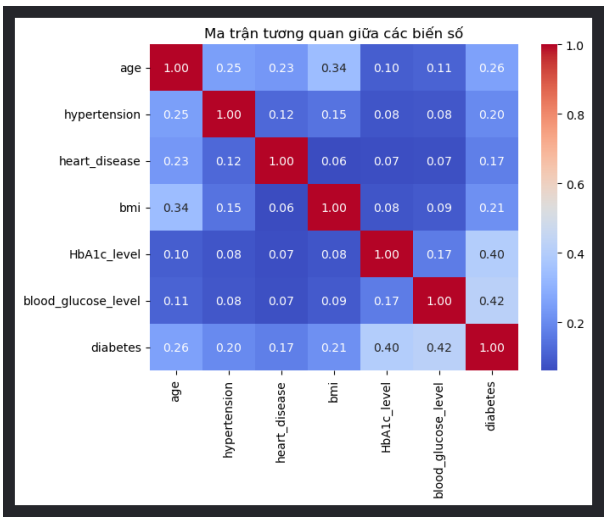
Tỷ lệ diabetes=1: 0.085

Phân bố nhãn:
diabetes
0    91500
1     8500
Name: count, dtype: int64
```

Hình 1.13. Thống kê mô tả và phân bố nhãn của dữ liệu tiểu đường

Bộ dữ liệu có 100.000 dòng và 9 cột, gồm 2 biến phân loại (gender, smoking_history) và các biến số age, hypertension, heart_disease, bmi, HbA1c_level, blood_glucose_level. Chỉ 8,5% mẫu mắc bệnh nên dữ liệu **mất cân bằng mạnh**: một mô hình luôn đoán "không mắc bệnh" đã đạt accuracy 91,5%. Vì vậy cần ưu tiên ROC AUC và recall lớp 1 hơn accuracy.

```
plt.figure(figsize=(7, 5))
numeric_corr = diab.select_dtypes(include='number').corr()
sns.heatmap(numeric_corr, annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Ma trận tương quan giữa các biến số'); plt.show()
```



Hình 1.14. Ma trận tương quan giữa các biến số

Hai biến tương quan mạnh nhất với diabetes là blood_glucose_level (0,42) và HbA1c_level (0,40), sau đó là age (0,26). Các biến độc lập tương quan yếu với nhau (cao nhất là age – bmi 0,34) nên không có hiện tượng đa cộng tuyến đáng kể.

1.4.2. Tiền xử lý dữ liệu

Dữ liệu không có giá trị thiếu. Hai thuộc tính phân loại được mã hóa One-Hot; cột diabetes được tách làm nhãn.

```
d2 = diab.copy()
y2 = d2.pop('diabetes')
X2 = pd.get_dummies(d2, columns=['gender', 'smoking_history'], dtype=int)
print('Kích thước X2:', X2.shape)
print('Số giá trị thiếu trong X2:', int(X2.isna().sum().sum()))
print('Các lớp của y2:', sorted(y2.unique()))
```

```
Kích thước X2: (100000, 15)
Số giá trị thiếu trong X2: 0
Các lớp của y2: [np.int64(0), np.int64(1)]
```

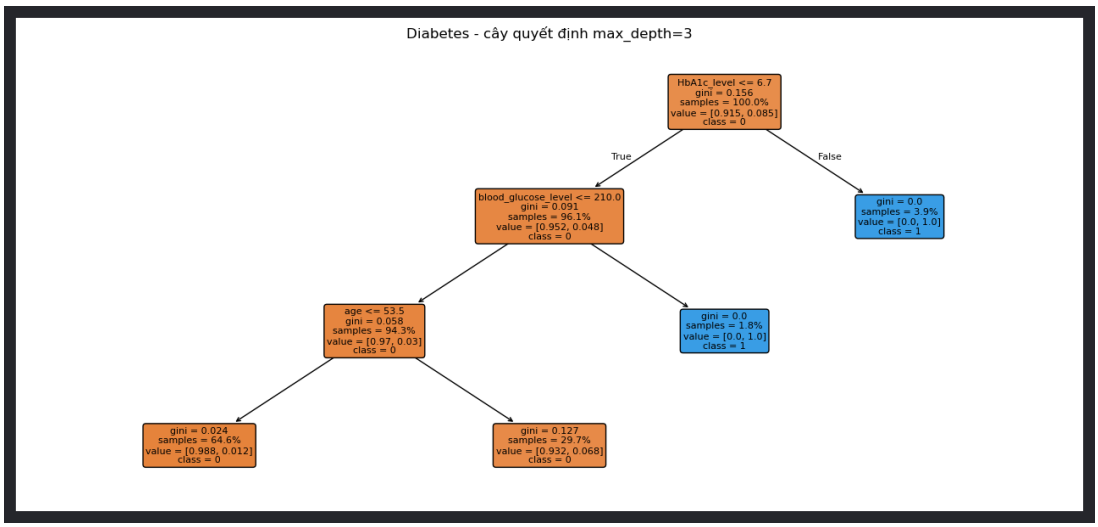
Hình 1.15. Kích thước dữ liệu sau mã hóa

1.4.3. Xây dựng, tối ưu và đánh giá mô hình

```
res2, imp2 = run_tree_forest(X2, y2, 'Diabetes')
res2
```

```
[Diabetes] train=(80000, 15), test=(20000, 15), tỷ lệ lớp 1 = 0.085
```

Hình 1.16. Kích thước tập train/test của dữ liệu tiểu đường

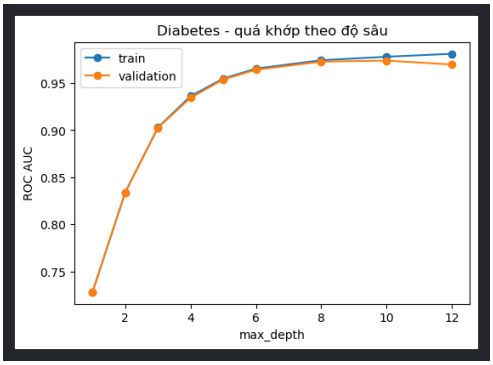


Hình 1.17. Cây quyết định max_depth=3 trên dữ liệu tiểu đường

Đọc cây: nút gốc là HbA1c_level <= 6.7. Toàn bộ người có HbA1c > 6.7 (3,9% mẫu) đều mắc bệnh (gini = 0). Với HbA1c <= 6.7, nếu blood_glucose_level > 210 thì cũng 100% mắc bệnh. Luật này khá khớp với tiêu chuẩn y khoa (HbA1c ≥ 6,5% là ngưỡng chẩn đoán tiểu đường).

```
Cây - tham số tốt nhất: {'max_depth': 10, 'min_samples_leaf': 10} | ROC AUC CV = 0.9738
```

Hình 1.18. Tham số tốt nhất của cây quyết định



Hình 1.19. ROC AUC train/validation theo độ sâu của cây (tiểu đường)

Khác với Titanic, hai đường train và validation gần như trùng nhau cho đến max_depth = 10 và chỉ tách nhẹ ở max_depth = 12. Nguyên nhân là tập train lớn (80.000 mẫu) cùng min_samples_leaf = 10 hạn chế cây học thuộc nhiễu. GridSearchCV chọn max_depth = 10, min_samples_leaf = 10 với ROC AUC CV = 0,974.

```
RF  - tham sô´tôt nhât: {'max_depth': 12, 'n_estimators': 200} | ROC AUC CV = 0.9724

=== Decision Tree ===
              precision    recall  f1-score   support

           0       0.97      1.00      0.98     18300
           1       0.97      0.69      0.80      1700

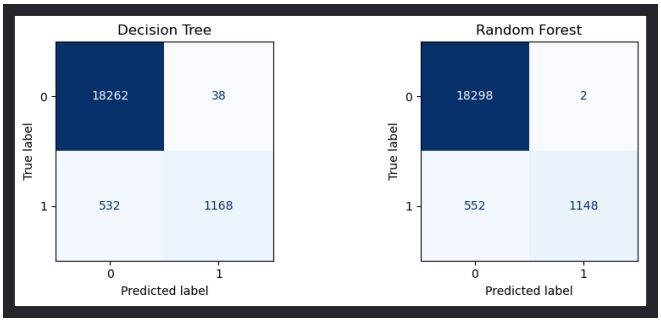
    accuracy                           0.97     20000
   macro avg       0.97      0.84      0.89     20000
weighted avg       0.97      0.97      0.97     20000


=== Random Forest ===
              precision    recall  f1-score   support

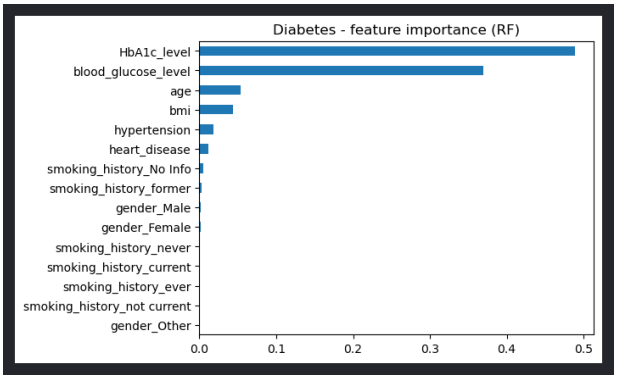
           0       0.97      1.00      0.99     18300
           1       1.00      0.68      0.81      1700

    accuracy                           0.97     20000
   macro avg       0.98      0.84      0.90     20000
weighted avg       0.97      0.97      0.97     20000
```

Hình 1.20. Tham số tốt nhất của Random Forest và báo cáo phân loại trên tập test



Hình 1.21. Ma trận nhầm lẫn của Decision Tree và Random Forest (tiểu đường)



Hình 1.22. Mức độ quan trọng của đặc trưng theo Random Forest (tiểu đường)

	Model	Accuracy test	ROC AUC train	ROC AUC test
0	Decision Tree	0.9715	0.9774	0.9746
1	Random Forest	0.9723	0.9824	0.9726

Hình 1.23. Bảng tổng hợp kết quả trên tập test (tiểu đường)

Nhận xét: hai mô hình cho kết quả rất sát nhau: accuracy ~0,972 và ROC AUC test ~0,973–0,975. Random Forest có precision lớp mắc bệnh gần như tuyệt đối (1,00, chỉ 2 trường hợp báo nhầm trên 20.000 mẫu), recall đạt 0,68; cây quyết định có recall 0,69. Trong bài toán y tế, recall lớp mắc bệnh có thể được nâng thêm bằng cách điều chỉnh ngưỡng phân lớp hoặc dùng class_weight='balanced'. HbA1c_level (~0,49) và

blood_glucose_level (~0,37) chiếm phần lớn độ quan trọng; giới tính và tiền sử hút thuốc gần như không đóng góp.

1.5. Nhận xét chương 1

Cây quyết định nông cho luật dễ đọc và khớp với hiểu biết thực tế (giới tính ở Titanic, HbA1c ở tiểu đường).

Đồ thị train/validation theo max_depth là công cụ hiệu quả để phát hiện quá khớp; dữ liệu nhỏ (Titanic) quá khớp nhanh hơn nhiều so với dữ liệu lớn (tiểu đường).

Random Forest cho ROC AUC ổn định và cao; với dữ liệu mất cân bằng, nhóm em đánh giá thêm recall, F1 bên cạnh accuracy để có cái nhìn đầy đủ.

Chương 2. SUPPORT VECTOR MACHINE (SVM)

2.1. Cơ sở lý thuyết

2.1.1. Siêu phẳng phân tách (hyperplane) và lề (margin)

SVM tìm một siêu phẳng $w \cdot x + b = 0$ để tách các điểm thuộc hai lớp (nhãn $y_i \in \{-1, +1\}$). Trong không gian 2 chiều siêu phẳng là một đường thẳng, trong 3 chiều là một mặt phẳng. Có vô số siêu phẳng tách được dữ liệu, SVM chọn siêu phẳng có **lề lớn nhất** – lề là khoảng cách từ siêu phẳng tới các điểm gần nhất của mỗi lớp, bằng $2/\|w\|$. Bài toán tối ưu (lề cứng):

$$\min \tfrac{1}{2}\|w\|^2 \quad \text{với ràng buộc} \quad y_i(w \cdot x_i + b) \geq 1, \ \forall i$$

Lề càng rộng thì mô hình càng ít nhạy với nhiễu và khả năng tổng quát hóa càng tốt. Khi dự đoán, mẫu mới được gán lớp theo dấu của $w \cdot x + b$.

2.1.2. Vai trò của các vector hỗ trợ (support vectors)

Support vectors là các điểm dữ liệu nằm đúng trên biên lề hoặc nằm trong lề/bị phân loại sai. Chỉ những điểm này quyết định vị trí của siêu phẳng; nếu bỏ các điểm khác đi thì kết quả không đổi. Chúng quan trọng vì: (1) định nghĩa ranh giới quyết định; (2) giúp mô hình gọn – hàm quyết định chỉ tính trên support vectors; (3) số support vectors phản ánh độ phức tạp – nếu tỷ lệ support vectors trên tập train quá lớn thì mô hình có xu hướng ghi nhớ dữ liệu.

2.1.3. Lề cứng (hard margin) và lề mềm (soft margin)

Lề cứng yêu cầu mọi điểm đều được phân loại đúng và nằm ngoài lề, chỉ dùng được khi dữ liệu tách được tuyến tính hoàn toàn và rất nhạy với ngoại lai. **Lề mềm** thêm biến bù $\xi_i \geq 0$ cho phép một số điểm vi phạm lề hoặc bị phân loại sai, đổi lại phải chịu phạt:

$$\min \tfrac{1}{2}\|w\|^2 + C \cdot \Sigma \xi_i \quad \text{với} \quad y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

Nên dùng lề mềm trong hầu hết bài toán thực tế vì dữ liệu thường có nhiễu, ngoại lai và các lớp chồng lấn. SVC của Scikit-learn mặc định là lề mềm.

2.1.4. Hàm nhân (kernel)

Khi dữ liệu không tách được tuyến tính, ta ánh xạ dữ liệu sang không gian nhiều chiều hơn $\varphi(x)$ để tách tuyến tính ở đó. "Kernel trick" cho phép tính trực tiếp tích vô hướng $K(x, x') = \varphi(x) \cdot \varphi(x')$ mà không cần tính φ tường minh. Các kernel phổ biến:

Linear: $K(x, x') = x \cdot x'$ – dùng khi dữ liệu gần tách tuyến tính hoặc số đặc trưng rất lớn (văn bản, dữ liệu One-Hot nhiều cột); nhanh, ít tham số.

Polynomial: $K(x, x') = (\gamma \cdot x \cdot x' + r)^d$ – mô hình tương tác giữa các đặc trưng đến bậc d; dễ quá khớp khi d lớn.

RBF (Gaussian): $K(x, x') = \exp(-\gamma\|x - x'\|^2)$ – linh hoạt nhất, là lựa chọn mặc định khi chưa biết cấu trúc dữ liệu và số đặc trưng không quá lớn.

Sigmoid: $K(x, x') = \tanh(\gamma \cdot x \cdot x' + r)$ – tương tự mạng nơ-ron một lớp, ít dùng trong thực tế.

2.1.5. Ý nghĩa của tham số C và gamma

C là hệ số phạt cho các vi phạm lề. C nhỏ → chấp nhận nhiều lỗi, lề rộng, mô hình đơn giản (có thể thiếu khớp). C lớn → phạt nặng lỗi, lề hẹp, cố phân loại đúng mọi điểm train (dễ quá khớp). **gamma** (với RBF, poly, sigmoid) quyết định phạm vi ảnh hưởng của mỗi điểm: gamma lớn → ranh giới uốn lượn sát dữ liệu (quá khớp), gamma nhỏ → ranh giới trơn. Hai tham số này thường được tìm đồng thời bằng GridSearchCV trên thang logarit.

2.1.6. Các bước xây dựng mô hình SVM bằng Scikit-learn

Bước 1: Nạp và khám phá dữ liệu, tách X và y.

Bước 2: Chia train/test bằng train_test_split (có stratify).

Bước 3: Xây dựng Pipeline gồm tiền xử lý (SimpleImputer, StandardScaler, OneHotEncoder trong ColumnTransformer) và SVC(kernel=…, C=…, gamma=…).

Bước 4: So sánh kernel bằng cross_val_score và tìm tham số tốt nhất với GridSearchCV.

Bước 5: Đánh giá trên tập test bằng accuracy, ROC AUC (SVC(probability=True) để có predict_proba), classification_report và confusion matrix.

2.1.7. Chuẩn hóa dữ liệu trước khi áp dụng SVM

Scikit-learn cung cấp **StandardScaler** (đưa về trung bình 0, độ lệch chuẩn 1), **MinMaxScaler** (đưa về [0, 1]) và **RobustScaler** (dùng trung vị và IQR, bền với ngoại lai). Bước này quan trọng vì SVM dựa trên khoảng cách và tích vô hướng: một đặc trưng có thang đo lớn (ví dụ blood_glucose_level 80–300) sẽ lấn át đặc trưng thang đo nhỏ (HbA1c 3,5–9), làm lề và kernel RBF bị méo; ngoài ra thuật toán tối ưu cũng hội tụ chậm hơn. Scaler phải được fit chỉ trên tập train (đặt trong Pipeline) để tránh rò rỉ dữ liệu sang tập test.

2.2. Thư viện và hàm dùng chung

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
import warnings; warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score, StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, roc_auc_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

import glob, os
def find_file(patterns):
    """Tìm file CSV ở thư mục hiện tại hoặc trong /kaggle/input (khi chạy trên Kaggle)."""
    for p in patterns:
        hits = glob.glob(p) + glob.glob('/kaggle/input/**/' + p, recursive=True)
        if hits: return hits[0]
    raise FileNotFoundError('Không thấy file ' + str(patterns) + ' - hãy tải từ Kaggle và đặt cạnh notebook.')
```

Hàm **run_svm(X, y, preprocessor, name)** thực hiện: chia train/test 80:20 (stratify); so sánh 4 kernel (linear, poly, rbf, sigmoid) với tham số mặc định bằng kiểm định chéo 5 lần (StratifiedKFold); GridSearchCV trên kernel, C, gamma (và degree cho poly); cuối cùng đánh giá một lần trên tập test, đồng thời so sánh điểm train/test để kiểm tra quá khớp.

```python
def run_svm(X, y, preprocessor, name):
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
    print(f'[{name}] train={X_train.shape}, test={X_test.shape}')
    cv = StratifiedKFold(5, shuffle=True, random_state=42)

    # 1) So sánh kernel với tham số mặc định
    rows = []
    for k in ['linear', 'poly', 'rbf', 'sigmoid']:
        pipe = Pipeline([('prep', preprocessor), ('svc', SVC(kernel=k))])
        sc = cross_val_score(pipe, X_train, y_train, cv=cv, scoring='accuracy')
        rows.append({'kernel': k, 'CV accuracy (mean)': sc.mean(), 'CV std': sc.std()})
    kdf = pd.DataFrame(rows).round(4); print(kdf.to_string(index=False))
    kdf.set_index('kernel')['CV accuracy (mean)'].plot.bar(figsize=(5, 3), ylim=(0, 1), rot=0)
    plt.title(f'{name} - so sánh kernel (CV accuracy)'); plt.show()

    # 2) GridSearchCV
    grid = GridSearchCV(Pipeline([('prep', preprocessor), ('svc', SVC(probability=True))]),
                        [{'svc__kernel': ['linear'], 'svc__C': [0.1, 1, 10, 100]},
                         {'svc__kernel': ['rbf'], 'svc__C': [0.1, 1, 10, 100], 'svc__gamma': ['scale', 0.001, 0.01, 0.1, 1]},
                         {'svc__kernel': ['poly'], 'svc__C': [0.1, 1, 10], 'svc__degree': [2, 3], 'svc__gamma': ['scale']}],
                        cv=cv, scoring='accuracy', n_jobs=-1, return_train_score=True)
    grid.fit(X_train, y_train)
    print('Tham số tốt nhất:', grid.best_params_)
    print('Accuracy CV: %.4f' % grid.best_score_)

    # 3) Đánh giá trên tập test
    best = grid.best_estimator_
    pred = best.predict(X_test); proba = best.predict_proba(X_test)[:, 1]
    print('Accuracy train: %.4f | Accuracy test: %.4f | ROC AUC test: %.4f' % (
        best.score(X_train, y_train), accuracy_score(y_test, pred), roc_auc_score(y_test, proba)))
    print(classification_report(y_test, pred))
    ConfusionMatrixDisplay(confusion_matrix(y_test, pred)).plot(cmap='Blues', colorbar=False)
    plt.title(f'{name} - confusion matrix (tập test)'); plt.show()
    return grid, (X_test, y_test)
```

2.3. Bài tập thực hành 1: Dữ liệu bệnh tiểu đường

2.3.1. Nạp dữ liệu và khám phá

Do SVM có độ phức tạp huấn luyện khoảng $O(n^2) – O(n^3)$ theo số mẫu, việc chạy GridSearchCV trên 100.000 dòng là không khả thi, nên notebook lấy mẫu phân tầng 1.000 dòng (giữ nguyên tỷ lệ 8,5% mắc bệnh).

```python
diab = pd.read_csv(find_file(['diabetes.csv', '*diabetes*.csv']))
target_col = next((col for col in ('Outcome', 'diabetes') if col in diab.columns), None)
if target_col is None:
    raise ValueError(f"Target column not found. Available columns: {diab.columns.tolist()}")

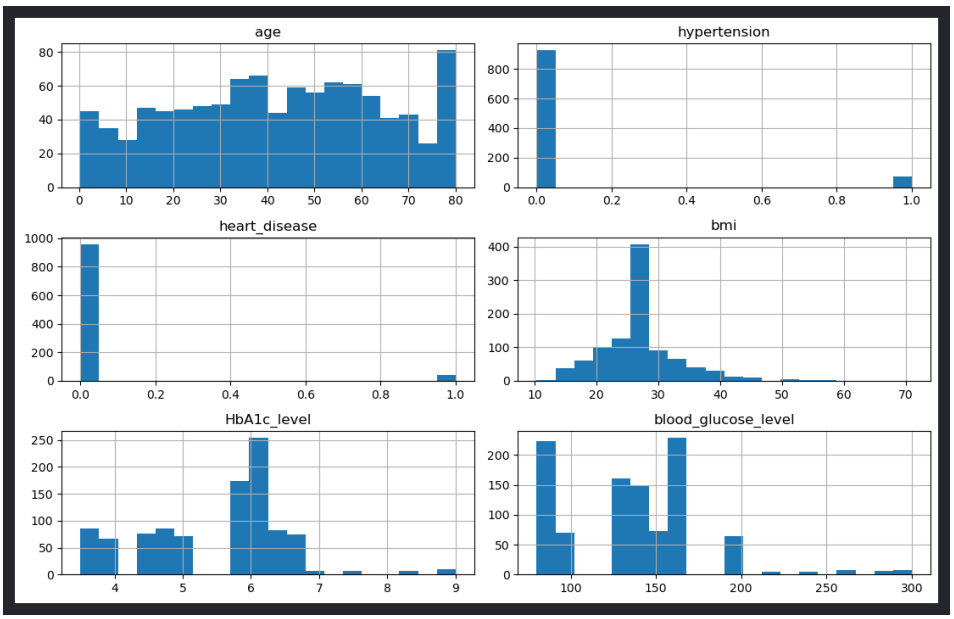
max_rows = 1000
if len(diab) > max_rows:
    diab, _ = train_test_split(diab, train_size=max_rows, stratify=diab[target_col], random_state=42)
    print(f"Using a stratified sample of {max_rows:,} rows to keep SVM search tractable.")

print(diab.shape, f'| target={target_col}, positive rate={diab[target_col].mean():.3f}')
diab.describe().T[['count', 'mean', 'std', 'min', 'max']]
```

```
Using a stratified sample of 1,000 rows to keep SVM search tractable.
(1000, 9) | target=diabetes, positive rate=0.085
                     count       mean        std    min     max
age                 1000.0   42.57632  22.303511   0.24   80.00
hypertension        1000.0    0.07500   0.263523   0.00    1.00
heart_disease       1000.0    0.04100   0.198389   0.00    1.00
bmi                 1000.0   27.20961   6.584692  10.30   70.96
HbA1c_level         1000.0    5.49270   1.053578   3.50    9.00
blood_glucose_level 1000.0  136.41700  40.229411  80.00  300.00
diabetes            1000.0    0.08500   0.279021   0.00    1.00
```

Hình 2.1. Thống kê mô tả của mẫu 1.000 dòng

```python
diab.drop(columns=target_col).select_dtypes(include=np.number).hist(bins=20, figsize=(11, 7)); plt.tight_layout(); plt.show()
```



Hình 2.2. Histogram các biến số của dữ liệu tiểu đường

Các biến có thang đo rất khác nhau (age 0–80, bmi 10–71, HbA1c 3,5–9, blood_glucose 80–300); hypertension và heart_disease là biến nhị phân lệch mạnh về 0. BMI tập trung đột biến quanh 27 (do bộ dữ liệu gốc điền giá trị thiếu bằng một hằng số), còn HbA1c và glucose nhận các giá trị rời rạc. Điều này khẳng định cần chuẩn hóa trước khi dùng SVM.

2.3.2. Tiền xử lý dữ liệu

Đoạn code được viết tổng quát: nếu dữ liệu có các cột Glucose, BloodPressure, SkinThickness, Insulin, BMI (như bộ Pima Indians Diabetes) thì giá trị 0 sẽ được coi là dữ liệu thiếu; bộ diabetes_prediction_dataset đang dùng đã đầy đủ dữ liệu nên giữ nguyên. Các bước tiền xử lý chính gồm: biến số → điền trung vị và StandardScaler; biến phân loại (gender, smoking_history) → điền mode và One-Hot, tất cả đặt trong ColumnTransformer để chỉ học trên tập train.

```
from sklearn.impute import SimpleImputer
d = diab.copy()
zero_missing = [col for col in ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI'] if col in d.columns]
d[zero_missing] = d[zero_missing].replace(0, np.nan)
X1, y1 = d.drop(columns=target_col), d[target_col]
numeric_cols = X1.select_dtypes(include=np.number).columns.tolist()
categorical_cols = X1.select_dtypes(exclude=np.number).columns.tolist()

def make_diabetes_preprocessor(scale_numeric=True):
    transformers = []
    if numeric_cols:
        numeric_steps = [('impute', SimpleImputer(strategy='median'))]
        if scale_numeric:
            numeric_steps.append(('scale', StandardScaler()))
        transformers.append(('num', Pipeline(numeric_steps), numeric_cols))
    if categorical_cols:
        categorical_steps = Pipeline([
            ('impute', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore')),
        ])
        transformers.append(('cat', categorical_steps, categorical_cols))
    return ColumnTransformer(transformers)

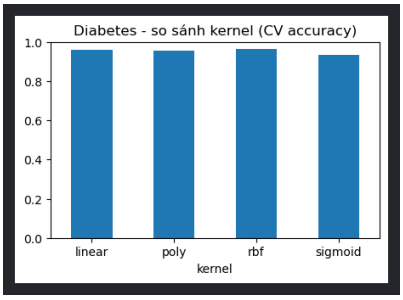
prep1 = make_diabetes_preprocessor()
```

2.3.3. Xây dựng, tối ưu và đánh giá SVM

```
grid1, test1 = run_svm(X1, y1, prep1, 'Diabetes')
```

```
[Diabetes] train=(800, 8), test=(200, 8)
 kernel  CV accuracy (mean)  CV std
 linear              0.9588  0.0116
   poly              0.9550  0.0092
    rbf              0.9625  0.0079
sigmoid              0.9325  0.0108
```

Hình 2.3. So sánh 4 kernel bằng CV accuracy (tiểu đường)



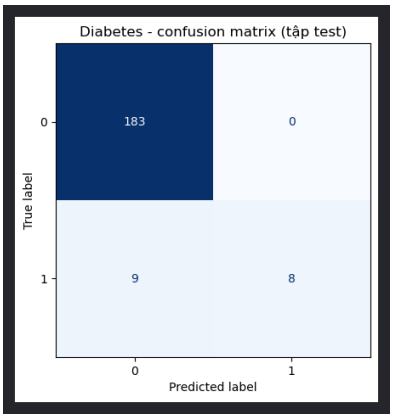
Hình 2.4. Biểu đồ CV accuracy theo kernel (tiểu đường)

```
Tham số tốt nhất: {'svc__C': 1, 'svc__gamma': 'scale', 'svc__kernel': 'rbf'}
Accuracy CV: 0.9625
Accuracy train: 0.9700 | Accuracy test: 0.9550 | ROC AUC test: 0.9132
              precision    recall  f1-score   support

           0       0.95      1.00      0.98       183
           1       1.00      0.47      0.64        17

    accuracy                           0.95       200
   macro avg       0.98      0.74      0.81       200
weighted avg       0.96      0.95      0.95       200
```

Hình 2.5. Tham số tốt nhất và kết quả trên tập test (tiểu đường)



Hình 2.6. Ma trận nhầm lẫn SVM trên tập test (tiểu đường)

Nhận xét: kernel RBF cho CV accuracy cao nhất (0,9625), linear và poly sát sau, sigmoid kém nhất (0,9325). GridSearchCV chọn RBF với C = 1, gamma = 'scale'. Accuracy train 0,970 và test 0,955 chênh lệch nhỏ nên mô hình không quá khớp. Mô hình đạt precision lớp mắc bệnh 1,00 (không có trường hợp báo nhầm) và ROC AUC 0,913. Do lớp mắc bệnh chỉ chiếm 8,5%, recall lớp này (0,47) có thể được cải thiện thêm bằng cách dùng scoring='roc_auc' hoặc 'f1', thêm class_weight='balanced' hoặc tăng kích thước mẫu.

2.3.4. Tác động của việc chuẩn hóa dữ liệu

```
Xtr, Xte, ytr, yte = train_test_split(X1, y1, test_size=0.2, random_state=42, stratify=y1)
no_scale = Pipeline([('prep', make_diabetes_preprocessor(scale_numeric=False)), ('svc', SVC(kernel='rbf'))]).fit(Xtr, ytr)
with_scale = Pipeline([('prep', make_diabetes_preprocessor(scale_numeric=True)), ('svc', SVC(kernel='rbf'))]).fit(Xtr, ytr)
print('Without numeric scaling: test accuracy = %.4f' % no_scale.score(Xte, yte))
print('With numeric scaling   : test accuracy = %.4f' % with_scale.score(Xte, yte))
```

```
Without numeric scaling: test accuracy = 0.9400
With numeric scaling   : test accuracy = 0.9550
```

Hình 2.7. Accuracy của SVM RBF khi có và không chuẩn hóa

Khi không chuẩn hóa, accuracy giảm từ 0,955 xuống 0,940 (tương đương 3 mẫu trên 200). Kết quả minh họa đúng lý thuyết: không chuẩn hóa thì blood_glucose_level chi phối khoảng cách trong kernel RBF.

2.4. Bài tập thực hành 2: Tình trạng động vật trong rừng

Mục tiêu: dự đoán con vật có ở tình trạng nguy hiểm (Dangerous: Yes/No) dựa trên tên loài (AnimalName) và 5 triệu chứng (symptoms1…symptoms5). Toàn bộ đặc trưng là phân loại nên phải mã hóa trước khi đưa vào SVM.

2.4.1. Nạp dữ liệu và khám phá

```
an = pd.read_csv(find_file(['animal_condition_train.csv', '*animal*.csv', 'data.csv']))
target = 'Dangerous' if 'Dangerous' in an.columns else an.columns[-1]
max_rows = 1000
if len(an) > max_rows:
    an, _ = train_test_split(an, train_size=max_rows, stratify=an[target], random_state=42)
    print(f"Using a stratified sample of {max_rows:,} rows to keep SVM search tractable.")
print(an.shape, '| label =', target)
print(an[target].value_counts(normalize=True).round(3))
print('Missing values:\n', an.isna().sum()[lambda s: s > 0])
an.head()
```

```
(871, 7) | label = Dangerous
Dangerous
Yes    0.977
No     0.023
Name: proportion, dtype: float64
Missing values:
 Dangerous    2
dtype: int64
  AnimalName symptoms1            symptoms2 symptoms3    symptoms4  \
0        Dog     Fever             Diarrhea  Vomiting  Weight loss
1        Dog     Fever             Diarrhea  Coughing    Tiredness
2        Dog     Fever             Diarrhea  Coughing     Vomiting
3        Dog     Fever  Difficulty breathing  Coughing     Lethargy
4        Dog     Fever             Diarrhea  Coughing     Lethargy

     symptoms5 Dangerous
0  Dehydration       Yes
1        Pains       Yes
2     Anorexia       Yes
3     Sneezing       Yes
4     Blue Eye       Yes
```

Hình 2.8. Kích thước, phân bố nhãn và 5 dòng đầu của dữ liệu động vật

Dữ liệu có 871 dòng, 7 cột. Nhãn mất cân bằng cực mạnh: 97,7% là Yes, chỉ 2,3% là No; cột Dangerous thiếu 2 giá trị.

2.4.2. Tiền xử lý dữ liệu

Các cột được điền thiếu bằng mode, sau đó mã hóa One-Hot với handle_unknown='ignore'. Nhóm em không dùng mã số nguyên 0, 1, 2… vì SVM sẽ hiểu nhầm là có thứ tự và khoảng cách giữa các triệu chứng. Dữ liệu One-Hot đã ở thang 0/1 nên không cần StandardScaler. Nhãn Yes/No chuyển thành 1/0.

```
from sklearn.impute import SimpleImputer
y2 = (an[target].astype(str).str.strip().str.lower() == 'yes').astype(int)
X2 = an.drop(columns=target).astype(str).replace('nan', np.nan)   # đảm bảo cột phân loại là chuỗi
cat_cols = X2.columns.tolist()
prep2 = ColumnTransformer([('cat', Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
                                             ('ohe', OneHotEncoder(handle_unknown='ignore'))]), cat_cols)])
print('Số đặc trưng sau One-Hot:', prep2.fit_transform(X2).shape[1])
```

```
Số đặc trưng sau One-Hot: 1157
```

Hình 2.9. Số đặc trưng sau One-Hot

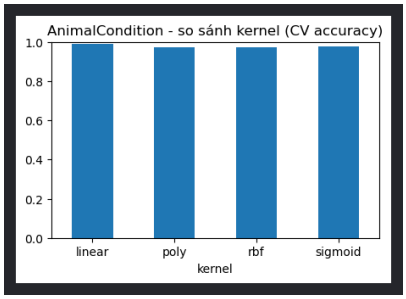
Sau mã hóa có tới 1.157 đặc trưng trong khi chỉ có 871 mẫu – số chiều lớn hơn số mẫu, là tình huống kernel linear thường phát huy hiệu quả.

2.4.3. Xây dựng, tối ưu và đánh giá SVM

```
grid2, test2 = run_svm(X2, y2, prep2, 'AnimalCondition')
```

```
[AnimalCondition] train=(696, 6), test=(175, 6)
 kernel  CV accuracy (mean)  CV std
 linear              0.9885  0.0057
   poly              0.9741  0.0035
    rbf              0.9741  0.0035
sigmoid              0.9785  0.0045
```

Hình 2.10. So sánh 4 kernel bằng CV accuracy (động vật)



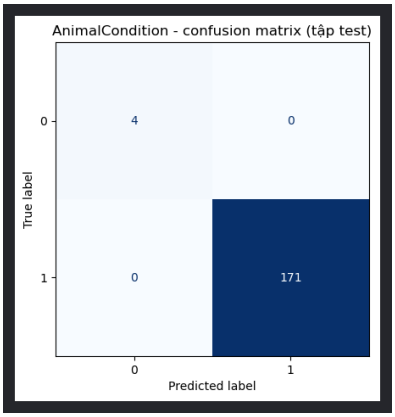
Hình 2.11. Biểu đồ CV accuracy theo kernel (động vật)

```
Tham số tốt nhất: {'svc__C': 1, 'svc__kernel': 'linear'}
Accuracy CV: 0.9885
Accuracy train: 1.0000 | Accuracy test: 1.0000 | ROC AUC test: 1.0000
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         4
           1       1.00      1.00      1.00       171

    accuracy                           1.00       175
   macro avg       1.00      1.00      1.00       175
weighted avg       1.00      1.00      1.00       175
```

Hình 2.12. Tham số tốt nhất và kết quả trên tập test (động vật)



Hình 2.13. Ma trận nhầm lẫn SVM trên tập test (động vật)

Nhận xét: đúng như lý thuyết, kernel linear tốt nhất (CV 0,9885) và được GridSearchCV chọn với C = 1. Trên tập test mô hình đạt accuracy, ROC AUC đều bằng 1,00. Mô hình phân loại đúng cả 4 mẫu lớp No và 171 mẫu lớp Yes, ma trận nhầm lẫn không có lỗi nào. Kết quả CV 0,9885 cũng cho thấy mô hình ổn định trên các phần dữ liệu khác nhau.

2.4.4. Số lượng support vectors

```
svc = grid2.best_estimator_.named_steps['svc']
print('Số support vectors theo lớp:', svc.n_support_, '| tổng =', svc.n_support_.sum(),
      '| tỷ lệ trên tập train = %.1f%%' % (100 * svc.n_support_.sum() / int(len(X2) * 0.8)))
```

```
Số support vectors theo lớp: [13 89] | tổng = 102 | tỷ lệ trên tập train = 14.7%
```

Hình 2.14. Số support vectors theo lớp

Mô hình dùng 102 support vectors (13 thuộc lớp No, 89 thuộc lớp Yes), chiếm 14,7% tập train. Tỷ lệ này khá thấp, cho thấy ranh giới phân lớp đơn giản và mô hình không ghi nhớ toàn bộ dữ liệu. Đáng chú ý là lớp No chỉ có khoảng 17 mẫu trong tập train nhưng có tới 13 mẫu là support vector, nghĩa là gần như mọi mẫu lớp thiểu số đều nằm sát ranh giới.

2.5. Nhận xét chương 2

Chuẩn hóa dữ liệu là bước bắt buộc với dữ liệu số khi dùng SVM; với dữ liệu One-Hot thì không cần.

Kernel RBF phù hợp với dữ liệu số ít chiều (tiểu đường), kernel linear phù hợp với dữ liệu nhiều chiều thưa (động vật, 1.157 đặc trưng).

Với dữ liệu lớn, việc lấy mẫu phân tầng giúp GridSearchCV cho SVM chạy trong thời gian hợp lý mà vẫn giữ tỷ lệ các lớp.

Chương 3. BAYES NGÂY THƠ (NAÏVE BAYES)

3.1. Cơ sở lý thuyết

3.1.1. Định lý Bayes và giả định "ngây thơ"

Naive Bayes là bộ phân lớp xác suất dựa trên định lý Bayes. Với mẫu $x = (x_1, \ldots, x_n)$ và lớp y:

$$P(y \mid x) = P(x \mid y) \cdot P(y) / P(x)$$

Trong đó P(y) là xác suất tiên nghiệm của lớp, P(x | y) là khả năng (likelihood), P(y | x) là xác suất hậu nghiệm. Ước lượng P(x | y) cho mọi tổ hợp đặc trưng là không khả thi, nên thuật toán đưa ra giả định "ngây thơ": **các đặc trưng độc lập có điều kiện với nhau khi biết lớp**. Khi đó:

$$\hat{y} = \text{argmax}_y \; P(y) \cdot \Pi \, P(x_i \mid y) \quad (\text{thực tế tính log: } \log P(y) + \Sigma \log P(x_i \mid y))$$

Mô hình chỉ cần học P(y) và từng $P(x_i \mid y)$ riêng lẻ từ dữ liệu train. Để tránh xác suất bằng 0 khi một giá trị chưa xuất hiện trong lớp, ta dùng làm trơn Laplace (tham số alpha).

3.1.2. Các loại mô hình Naive Bayes

GaussianNB: giả sử mỗi đặc trưng liên tục trong mỗi lớp tuân theo phân phối chuẩn $N(\mu, \sigma^2)$; dùng cho dữ liệu số liên tục như tuổi, thu nhập.

MultinomialNB: $P(x_i \mid y)$ theo phân phối đa thức trên số đếm; dùng cho đặc trưng đếm/tần suất không âm, điển hình là số lần xuất hiện từ trong văn bản (CountVectorizer, TF-IDF).

BernoulliNB: mỗi đặc trưng là nhị phân 0/1 (có/không xuất hiện); dùng cho văn bản ngắn hoặc dữ liệu One-Hot; khác Multinomial ở chỗ có tính cả xác suất "vắng mặt".

CategoricalNB: mỗi đặc trưng là biến phân loại nhận k giá trị rời rạc, học trực tiếp P(giá trị | lớp); phù hợp nhất với dữ liệu thuần phân loại như Mushroom.

3.1.3. Vì sao gọi là "ngây thơ" và ảnh hưởng của giả định độc lập

Gọi là "ngây thơ" vì trong thực tế các đặc trưng hiếm khi độc lập (ví dụ tuổi và thu nhập có tương quan, màu mũ và màu thân nấm liên quan nhau). Khi các đặc trưng tương quan, mô hình "đếm" trùng bằng chứng nhiều lần nên xác suất dự đoán thường bị đẩy về gần 0 hoặc 1 (kém hiệu chỉnh). Tuy vậy, để phân lớp đúng chỉ cần lớp đúng có điểm cao nhất, nên Naive Bayes vẫn thường cho kết quả tốt, đặc biệt khi dữ liệu ít hoặc số chiều cao; nó chỉ kém rõ rệt khi tương quan giữa các đặc trưng quyết định nhãn.

3.1.4. Ưu điểm và hạn chế so với SVM và Random Forest

Ưu điểm: huấn luyện và dự đoán rất nhanh (một lần duyệt dữ liệu), cần ít dữ liệu, hoạt động tốt với số chiều lớn (văn bản), ít tham số, ít quá khớp, cho xác suất trực tiếp và dễ giải thích từng đặc trưng. **Hạn chế:** giả định độc lập thường bị vi phạm nên độ chính xác thường thấp hơn SVM hoặc Random Forest khi dữ liệu đủ lớn và có tương tác phức tạp; xác suất đầu ra kém hiệu chỉnh; GaussianNB nhạy với đặc trưng không có dạng chuẩn; không học được tương tác giữa các đặc trưng như cây hay kernel.

3.1.5. Các bước xây dựng mô hình Naive Bayes bằng Scikit-learn

Bước 1: Nạp dữ liệu, bỏ cột định danh, mã hóa biến phân loại.

Bước 2: Chia train/test bằng train_test_split (stratify).

Bước 3: Khởi tạo mô hình phù hợp với kiểu dữ liệu, ví dụ GaussianNB(), rồi gọi .fit(X_train, y_train). Có thể xem các tham số đã học: class_prior_, theta_, var_.

Bước 4: Dự đoán bằng .predict()/.predict_proba() và đánh giá bằng accuracy, ROC AUC, classification_report, confusion matrix; kiểm định chéo bằng cross_val_score.

3.1.6. Xử lý dữ liệu phân loại trước khi dùng Multinomial Naive Bayes

MultinomialNB yêu cầu đặc trưng không âm dạng đếm, nên dữ liệu phân loại cần được mã hóa **One-Hot** (OneHotEncoder hoặc pd.get_dummies) – mỗi giá trị thành một cột 0/1. Không nên dùng LabelEncoder/OrdinalEncoder cho MultinomialNB vì số nguyên sẽ bị hiểu như số đếm (giá trị 5 "gấp 5 lần" giá trị 1). Ngược lại, OrdinalEncoder phù hợp với CategoricalNB vì mô hình này coi số nguyên chỉ là chỉ số của giá trị. Giá trị thiếu nên điền bằng mode hoặc giữ thành một giá trị riêng (ví dụ "Unknown").

3.1.7. Naive Bayes trong phân loại văn bản

Quy trình gồm: (1) tiền xử lý văn bản – chuyển chữ thường, bỏ dấu câu, stop words; (2) vector hóa bằng CountVectorizer (túi từ) hoặc TfidfVectorizer, fit trên tập train và chỉ transform tập test; (3) huấn luyện MultinomialNB (hoặc ComplementNB khi lớp mất cân bằng), mỗi từ là một đặc trưng và P(từ | lớp) được ước lượng từ tần suất có làm trơn Laplace; (4) đánh giá bằng precision, recall, F1. Naive Bayes rất hợp với văn bản vì số chiều lớn, dữ liệu thưa và giả định độc lập giữa các từ tuy không đúng nhưng vẫn cho kết quả tốt, ví dụ lọc thư rác, phân loại cảm xúc.

3.2. Thư viện sử dụng

Ngoài các thư viện xử lý và trực quan hóa, nhóm em dùng bốn biến thể GaussianNB, CategoricalNB, BernoulliNB, MultinomialNB cùng các bộ mã hóa StandardScaler, OrdinalEncoder, OneHotEncoder.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns
import warnings; warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.naive_bayes import GaussianNB, CategoricalNB, BernoulliNB, MultinomialNB
from sklearn.preprocessing import StandardScaler, OrdinalEncoder, OneHotEncoder
from sklearn.metrics import (accuracy_score, roc_auc_score, classification_report,
                             confusion_matrix, ConfusionMatrixDisplay)

import glob, os
def find_file(patterns):
    """Tìm file CSV ở thư mục hiện tại hoặc trong /kaggle/input (khi chạy trên Kaggle)."""
    for p in patterns:
        hits = glob.glob(p) + glob.glob('/kaggle/input/**/' + p, recursive=True)
        if hits: return hits[0]
    raise FileNotFoundError('Không thấy file ' + str(patterns) + ' - hãy tải từ Kaggle và đặt cạnh notebook.')
```

3.3. Bài tập thực hành 1: Dự đoán hành vi khách hàng

Mục tiêu: dự đoán người dùng mạng xã hội có mua sản phẩm sau khi xem quảng cáo (Purchased = 1) dựa trên tuổi (Age), thu nhập ước tính (EstimatedSalary) và giới tính (Gender).

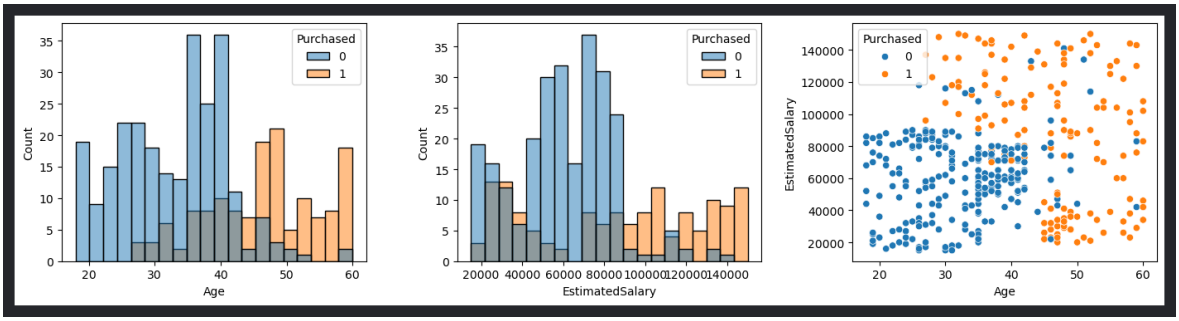
3.3.1. Nạp dữ liệu và khám phá

```
ads = pd.read_csv(find_file(['Customer_Behaviour.csv']))
print(ads.shape, '| thiếu:', int(ads.isna().sum().sum()), '| tỷ lệ mua: %.3f' % ads['Purchased'].mean())
ads.head()
```

```
(400, 5) | thiếu: 0 | tỷ lệ mua: 0.357
    User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0
```

Hình 3.1. Kích thước và 5 dòng đầu của dữ liệu khách hàng

```
fig, axs = plt.subplots(1, 3, figsize=(14, 3.6))
sns.histplot(data=ads, x='Age', hue='Purchased', bins=20, ax=axs[0])
sns.histplot(data=ads, x='EstimatedSalary', hue='Purchased', bins=20, ax=axs[1])
sns.scatterplot(data=ads, x='Age', y='EstimatedSalary', hue='Purchased', ax=axs[2])
plt.tight_layout(); plt.show()
```



Hình 3.2. Phân bố Age, EstimatedSalary theo Purchased và biểu đồ phân tán

Dữ liệu gồm 400 khách hàng, không có giá trị thiếu, 35,7% đã mua hàng. Người mua tập trung ở nhóm tuổi trên 45 hoặc thu nhập trên ~90.000; biểu đồ phân tán cho thấy ranh giới giữa hai lớp không phải đường thẳng mà có dạng cong.

3.3.2. Tiền xử lý và huấn luyện GaussianNB

Cột User ID bị loại vì chỉ là định danh; Gender được chuyển thành 0/1. Dữ liệu chia 75:25 (stratify). GaussianNB học cho mỗi lớp trung bình và phương sai của từng đặc trưng (theta_, var_) cùng xác suất tiên nghiệm (class_prior_).

```
d = ads.drop(columns=[c for c in ['User ID', 'UserID'] if c in ads.columns]).copy()
if 'Gender' in d.columns: d['Gender'] = (d['Gender'].astype(str).str.lower() == 'male').astype(int)
X1, y1 = d.drop(columns='Purchased'), d['Purchased']
X_train, X_test, y_train, y_test = train_test_split(X1, y1, test_size=0.25, random_state=0, stratify=y1)

gnb = GaussianNB().fit(X_train, y_train)
print('Xác suất tiên nghiệm P(lớp):', gnb.class_prior_.round(3))
print(pd.DataFrame(gnb.theta_, columns=X1.columns, index=['lớp 0', 'lớp 1']).round(2))
```

```
Xác suất tiên nghiệm P(lớp): [0.643 0.357]
        Gender    Age  EstimatedSalary
lớp 0     0.49  32.82         59585.49
lớp 1     0.47  46.47         87345.79
```

Hình 3.3. Xác suất tiên nghiệm và trung bình theo lớp

Xác suất tiên nghiệm là 0,643 / 0,357, đúng bằng tỷ lệ lớp trong tập train. Trung bình tuổi của người mua (46,5) cao hơn hẳn người không mua (32,8), thu nhập trung

bình cũng cao hơn (87.346 so với 59.585), còn Gender gần như bằng nhau giữa hai lớp (0,49 và 0,47) nên hầu như không có khả năng phân biệt.

3.3.3. Đánh giá mô hình

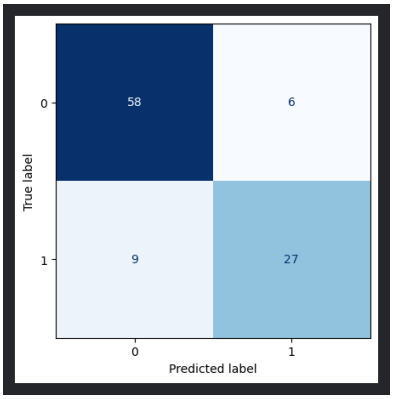
```
pred = gnb.predict(X_test); proba = gnb.predict_proba(X_test)[:, 1]
print('Accuracy: %.4f | ROC AUC: %.4f' % (accuracy_score(y_test, pred), roc_auc_score(y_test, proba)))
print(classification_report(y_test, pred))
ConfusionMatrixDisplay(confusion_matrix(y_test, pred)).plot(cmap='Blues', colorbar=False); plt.show()
```

```
Accuracy: 0.8500 | ROC AUC: 0.9462
              precision    recall  f1-score   support

           0       0.87      0.91      0.89        64
           1       0.82      0.75      0.78        36

    accuracy                           0.85       100
   macro avg       0.84      0.83      0.83       100
weighted avg       0.85      0.85      0.85       100
```

Hình 3.4. Accuracy, ROC AUC và báo cáo phân loại GaussianNB



Hình 3.5. Ma trận nhầm lẫn GaussianNB (khách hàng)

Mô hình đạt accuracy 0,85 và ROC AUC 0,946 trên 100 mẫu test. Lớp 0 có recall 0,91, lớp 1 có precision 0,82 và recall 0,75. Với một mô hình rất đơn giản chỉ có 3 đặc trưng, đây là kết quả khá tốt.

3.3.4. So sánh có/không chuẩn hóa và kiểm định chéo

```
cv = StratifiedKFold(5, shuffle=True, random_state=0)
raw = cross_val_score(GaussianNB(), X1, y1, cv=cv).mean()
sc = StandardScaler().fit(X_train)
scaled = GaussianNB().fit(sc.transform(X_train), y_train).score(sc.transform(X_test), y_test)
print('CV accuracy (dữ liệu gốc)   : %.4f' % raw)
print('Test accuracy (đã chuẩn hóa): %.4f  (so với %.4f khi chưa chuẩn hóa)' % (scaled, accuracy_score(y_test, pred)))
```

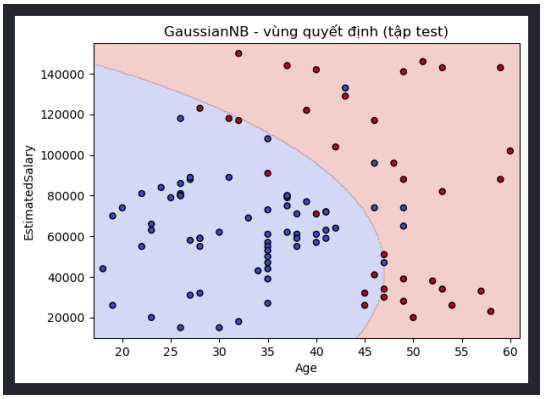
```
CV accuracy (dữ liệu gốc)   : 0.8800
Test accuracy (đã chuẩn hóa): 0.8500  (so với 0.8500 khi chưa chuẩn hóa)
```

Hình 3.6. Kết quả kiểm định chéo và so sánh chuẩn hóa

Accuracy trên tập test giữ nguyên 0,85 dù có chuẩn hóa hay không. Điều này đúng với lý thuyết: GaussianNB mô hình hóa từng đặc trưng bằng một phân phối chuẩn riêng, phép biến đổi tuyến tính chỉ dịch chuyển và co giãn μ, σ nên không làm thay đổi quyết định. CV accuracy 5 lần trên toàn bộ dữ liệu là 0,88, cao hơn test một chút, cho thấy kết quả test 0,85 nằm trong mức dao động bình thường.

3.3.5. Ranh giới quyết định

```
cols = ['Age', 'EstimatedSalary']
m2 = GaussianNB().fit(X_train[cols], y_train)
xx, yy = np.meshgrid(np.linspace(X1[cols[0]].min() - 1, X1[cols[0]].max() + 1, 300),
                     np.linspace(X1[cols[1]].min() - 5000, X1[cols[1]].max() + 5000, 300))
zz = m2.predict(pd.DataFrame(np.c_[xx.ravel(), yy.ravel()], columns=cols)).reshape(xx.shape)
plt.figure(figsize=(6.5, 4.5)); plt.contourf(xx, yy, zz, alpha=0.25, cmap='coolwarm')
plt.scatter(X_test[cols[0]], X_test[cols[1]], c=y_test, cmap='coolwarm', edgecolor='k', s=25)
plt.xlabel(cols[0]); plt.ylabel(cols[1]); plt.title('GaussianNB - vùng quyết định (tập test)'); plt.show()
```



Hình 3.7. Vùng quyết định của GaussianNB trên mặt phẳng Age – EstimatedSalary

Ranh giới quyết định có dạng đường cong bậc hai do phương sai của hai lớp khác nhau. Vùng màu xanh (không mua) bao quanh nhóm người trẻ, thu nhập trung bình; phần lớn điểm bị phân loại sai nằm sát ranh giới ở vùng tuổi 40–50.

3.4. Bài tập thực hành 2: Phân loại nấm (Mushroom)

Mục tiêu: phân loại nấm ăn được (e) hay độc (p) từ 22 đặc trưng mô tả hình dạng, màu sắc, mùi… Toàn bộ đặc trưng là phân loại nên cần mã hóa rồi dùng các biến thể Naive Bayes cho dữ liệu rời rạc.

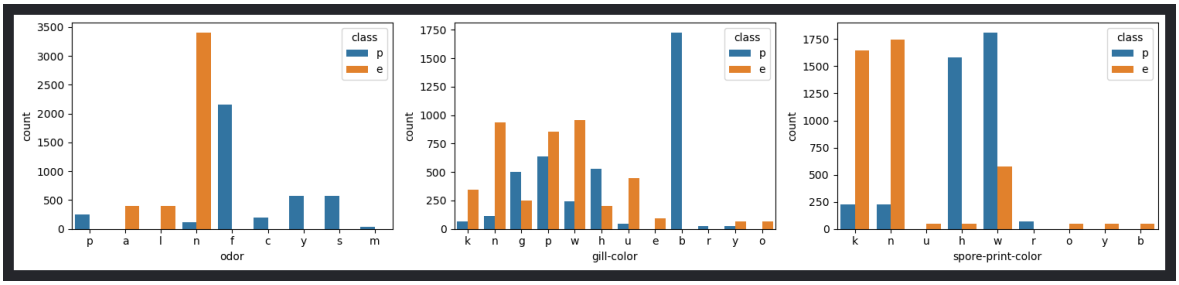
3.4.1. Nạp dữ liệu và khám phá

```
mush = pd.read_csv(find_file(['mushrooms.csv', 'mushroom*.csv', '*mushroom*.csv']))
print(mush.shape)
print(mush['class'].value_counts(normalize=True).round(3))
print('Số giá trị khác nhau mỗi cột:'); print(mush.nunique().sort_values(ascending=False).head(8).to_string())
print('Cột có ký tự "?" (dữ liệu thiếu):', [c for c in mush.columns if (mush[c] == '?').any()])
```

```
(8124, 23)
class
e    0.518
p    0.482
Name: proportion, dtype: float64
Số giá trị khác nhau mỗi cột:
gill-color                12
cap-color                 10
spore-print-color          9
odor                       9
stalk-color-below-ring     9
stalk-color-above-ring     9
habitat                    7
cap-shape                  6
Cột có ký tự "?" (dữ liệu thiếu): ['stalk-root']
```

Hình 3.8. Phân bố nhãn và số giá trị khác nhau của các cột

```
top = [c for c in ['odor', 'gill-color', 'spore-print-color'] if c in mush.columns][:3] or mush.columns[1:4].tolist()
fig, axs = plt.subplots(1, len(top), figsize=(5 * len(top), 3.4))
for ax, c in zip(np.atleast_1d(axs), top):
    sns.countplot(data=mush, x=c, hue='class', ax=ax)
plt.tight_layout(); plt.show()
```



Hình 3.9. Phân bố odor, gill-color, spore-print-color theo lớp nấm

Dữ liệu có 8.124 mẫu, hai lớp khá cân bằng (51,8% ăn được, 48,2% độc). Cột stalk-root chứa ký tự "?" biểu thị dữ liệu thiếu. Biểu đồ cho thấy odor gần như tách hoàn toàn hai lớp: nấm không mùi (n), mùi hồi (a), mùi hạnh nhân (l) phần lớn ăn được; mùi hôi (f), cay (y, s) đều là nấm độc. Gill-color = b (buff) cũng chỉ xuất hiện ở nấm độc.

3.4.2. Tiền xử lý dữ liệu

Nhãn p (độc) = 1 vì đây là lớp cần phát hiện. Ký tự "?" được giữ như một giá trị riêng. Nhóm em dùng hai cách mã hóa: **Ordinal** cho CategoricalNB (số nguyên chỉ là chỉ số giá trị) và **One-Hot** cho BernoulliNB, MultinomialNB (yêu cầu đặc trưng không âm). Các bộ mã hóa học danh sách giá trị trên toàn bộ X (không dùng nhãn nên không rò rỉ) để tránh gặp giá trị lạ ở tập test.

```
y2 = (mush['class'] == 'p').astype(int)
X2 = mush.drop(columns='class')
ord_enc = OrdinalEncoder().fit(X2)
X_ord = pd.DataFrame(ord_enc.transform(X2).astype(int), columns=X2.columns)
n_cat = [len(c) for c in ord_enc.categories_]
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False).fit(X2)
X_ohe = ohe.transform(X2)
print('Ordinal:', X_ord.shape, '| One-Hot:', X_ohe.shape)

idx_tr, idx_te = train_test_split(np.arange(len(X2)), test_size=0.2, random_state=42, stratify=y2)
ytr, yte = y2.iloc[idx_tr], y2.iloc[idx_te]
```

```
Ordinal: (8124, 22) | One-Hot: (8124, 117)
```

Hình 3.10. Kích thước dữ liệu sau hai cách mã hóa

3.4.3. Huấn luyện và so sánh 3 mô hình

```
models = {
    'CategoricalNB (ordinal)': (CategoricalNB(min_categories=n_cat), X_ord.values),
    'BernoulliNB (one-hot)':   (BernoulliNB(), X_ohe),
    'MultinomialNB (one-hot)': (MultinomialNB(), X_ohe),
}
rows, preds = [], {}
for name, (clf, Xm) in models.items():
    clf.fit(Xm[idx_tr], ytr); p = clf.predict(Xm[idx_te]); preds[name] = p
    tn, fp, fn, tp = confusion_matrix(yte, p).ravel()
    rows.append({'Mô hình': name, 'Accuracy': accuracy_score(yte, p),
                 'ROC AUC': roc_auc_score(yte, clf.predict_proba(Xm[idx_te])[:, 1]),
                 'Độc bị đoán là ăn được (FN)': fn, 'Ăn được bị đoán là độc (FP)': fp})
pd.DataFrame(rows).round(4)
```

```
                  Mô hình  Accuracy  ROC AUC  Độc bị đoán là ăn được (FN)  \
0  CategoricalNB (ordinal)    0.9458   0.9972                           81
1    BernoulliNB (one-hot)    0.9329   0.9954                           96
2  MultinomialNB (one-hot)    0.9458   0.9972                           81

   Ăn được bị đoán là độc (FP)
0                            7
1                           13
2                            7
```

Hình 3.11. So sánh CategoricalNB, BernoulliNB và MultinomialNB

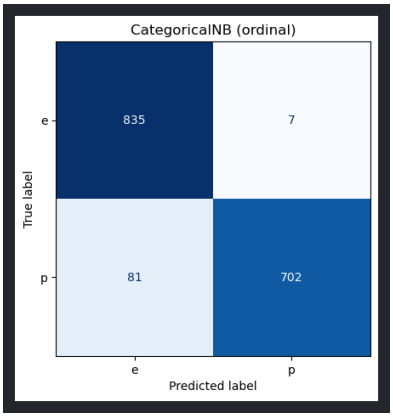
```
best = max(models, key=lambda k: accuracy_score(yte, preds[k]))
print('Mô hình tốt nhất theo accuracy:', best)
print(classification_report(yte, preds[best], target_names=['ăn được (e)', 'độc (p)']))
ConfusionMatrixDisplay(confusion_matrix(yte, preds[best]), display_labels=['e', 'p']).plot(cmap='Blues', colorbar=False)
plt.title(best); plt.show()
```

```
Mô hình tốt nhất theo accuracy: CategoricalNB (ordinal)
              precision    recall  f1-score   support

  ăn được (e)       0.91      0.99      0.95       842
     độc (p)       0.99      0.90      0.94       783

    accuracy                           0.95      1625
   macro avg       0.95      0.94      0.95      1625
weighted avg       0.95      0.95      0.95      1625
```

Hình 3.12. Báo cáo phân loại của mô hình tốt nhất



Hình 3.13. Ma trận nhầm lẫn CategoricalNB (nấm)

Nhận xét: CategoricalNB và MultinomialNB cho kết quả giống hệt nhau (accuracy 0,9458, ROC AUC 0,9972), BernoulliNB thấp hơn (0,9329). Điều này hợp lý vì với dữ liệu One-Hot mỗi cột chỉ có đúng một giá trị 1, MultinomialNB ước lượng xác suất tương đương CategoricalNB. ROC AUC rất cao cho thấy mô hình xếp hạng gần như hoàn hảo; recall lớp độc đạt 0,90. Precision lớp độc đạt 0,99, tức khi mô hình kết luận nấm độc thì gần như chắc chắn đúng. Trong ứng dụng thực tế, recall lớp độc có thể được nâng thêm bằng cách hạ ngưỡng xác suất phân lớp.

3.4.4. Đặc trưng phân biệt nấm độc

Với CategoricalNB, nhóm em dùng chênh lệch log P(giá trị | độc) − log P(giá trị | ăn được) để xem giá trị nào nghiêng mạnh về mỗi lớp.

```
cnb = models['CategoricalNB (ordinal)'][0]
rows = []
for j, col in enumerate(X2.columns):
    lp = cnb.feature_log_prob_[j]                    # (số lớp, số giá trị)
    for k, val in enumerate(ord_enc.categories_[j]):
        rows.append({'đặc trưng': col, 'giá trị': val, 'log P(v|độc) - log P(v|ăn được)': lp[1, k] - lp[0, k]})
d = pd.DataFrame(rows)
print('Nghiêng mạnh về NẤM ĐỘC:');      print(d.nlargest(8, d.columns[-1]).round(2).to_string(index=False))
print('\nNghiêng mạnh về ĂN ĐƯỢC:');   print(d.nsmallest(8, d.columns[-1]).round(2).to_string(index=False))
```

```
Nghiêng mạnh về NẤM ĐỘC:
             đặc trưng giá trị  log P(v|độc) - log P(v|ăn được)
                  odor       f                             7.54
            gill-color       b                             7.29
             ring-type       l                             7.03
                  odor       y                             6.20
                  odor       s                             6.18
stalk-color-above-ring       b                             5.94
stalk-color-below-ring       b                             5.94
                  odor       p                             5.34

Nghiêng mạnh về ĂN ĐƯỢC:
             đặc trưng giá trị  log P(v|độc) - log P(v|ăn được)
stalk-color-below-ring       g                            -6.08
stalk-color-above-ring       g                            -6.07
            population       n                            -5.74
                  odor       l                            -5.70
                  odor       a                            -5.69
            population       a                            -5.65
               habitat       w                            -4.99
            stalk-root       r                            -4.96
```

Hình 3.14. Các giá trị đặc trưng nghiêng mạnh về nấm độc và nấm ăn được

Các dấu hiệu mạnh nhất của nấm độc là odor = f (hôi), gill-color = b, ring-type = l (vòng lớn), odor = y/s (cay) – chênh lệch log trên 6, tức khả năng xuất hiện ở nấm độc lớn hơn hàng trăm lần. Ngược lại, stalk-color = g (xám), population = n/a, odor = l/a (hạnh nhân, hồi) nghiêng mạnh về nấm ăn được. Kết quả khớp với biểu đồ khám phá ở mục 3.4.1.

3.5. Nhận xét chương 3

Cần chọn biến thể Naive Bayes theo kiểu dữ liệu: Gaussian cho biến liên tục, Categorical/Multinomial/Bernoulli cho biến rời rạc.

GaussianNB không bị ảnh hưởng bởi chuẩn hóa tuyến tính, khác với SVM.

Naive Bayes huấn luyện rất nhanh, dễ diễn giải và vẫn đạt accuracy khoảng 0,95, ROC AUC 0,997 trên dữ liệu Mushroom.

KẾT LUẬN

Qua bài lab, nhóm em đã cài đặt và đánh giá ba nhóm thuật toán phân lớp cơ bản trên sáu bộ dữ liệu thực tế. Một số kết luận chính:

Cây quyết định và Random Forest dễ dùng, không cần chuẩn hóa, cho luật và mức độ quan trọng của đặc trưng dễ giải thích. Random Forest có ROC AUC ổn định hơn cây đơn nhưng vẫn cần kiểm soát độ sâu để tránh quá khớp.

SVM mạnh trên dữ liệu số đã chuẩn hóa (kernel RBF) và dữ liệu nhiều chiều (kernel linear), nhưng tốn chi phí tính toán lớn nên phải lấy mẫu khi dữ liệu lớn.

Naive Bayes rất nhanh và hiệu quả với ít dữ liệu, nhưng giả định độc lập làm giảm độ chính xác khi các đặc trưng tương quan.

Với các bộ **dữ liệu mất cân bằng** (tiểu đường 8,5%, động vật 2,3% lớp No), nhóm em đánh giá kết hợp accuracy, recall, F1 và ROC AUC để có cái nhìn toàn diện.

Hướng phát triển: nhóm em sẽ tiếp tục thử nghiệm class_weight='balanced', điều chỉnh ngưỡng phân lớp, kỹ thuật SMOTE và so sánh thêm các mô hình khác để nâng cao hơn nữa hiệu quả dự đoán.

TÀI LIỆU THAM KHẢO

[1] Trường Đại học Sài Gòn, Tài liệu thực hành "Các giải thuật phân loại cơ bản", Khoa Công nghệ Thông tin.

[2] Scikit-learn developers, "Scikit-learn User Guide: Decision Trees, Ensemble methods, Support Vector Machines, Naive Bayes", https://scikit-learn.org/stable/user_guide.html.

[3] Kaggle, Titanic – Machine Learning from Disaster; Diabetes Prediction Dataset; Animal Condition Classification; Customer Behaviour (Social Network Ads); Mushroom Classification (UCI).

[4] Géron, A., Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed., O'Reilly, 2022.

[5] Chapman, P. et al., CRISP-DM 1.0: Step-by-step data mining guide, SPSS, 2000.